Trabajo de SED

FPGA's

Grupo 10

Participantes

Guillermo Rosales Pérez	**Nº 56070**
Lorenzo Hidalgo Anero	**Nº 56415**
Matías López Viagel	**Nº 56444**

Índice

1. Introducción y objetivos 3
1.1. Tarea a realizar 3
1.2. Objetivos 3
2. Descripción del proyecto 4
2.1. Funcionamiento 4
2.2. Entradas, salidas y constraints 5
2.2.1. Entradas 5
2.2.2. Salidas 5
2.2.3. Constraints 5
2.3. Diagrama de estados 6
3. Desarrollo de los componentes del código y simulación 7
3.1. Relación y comunicación entre los bloques 7
3.2. Esquema de conexión 8
3.3. Bloques 10
3.3.1. Prescaler 10
3.3.1.1. CÓDIGO 10
3.3.1.2. TESTBENCH 11
3.3.2. Sincronizador 12
3.3.2.1. CÓDIGO 12
3.3.2.2.TESTBENCH 14
3.3.3. EDGE_DETECT 14
3.3.3.1.CÓDIGO 15
3.3.3.2.TESTBENCH 18
3.3.4. Maq_Estados 19
3.3.4.1. FOTOS DEL PROCESO 20
3.3.4.2.CÓDIGO 21
3.3.4.3.TESTBENCH 25
3.3.5. Contador 25
3.3.5.1.CÓDIGO 26
3.3.5.2.TESTBENCH 30
3.3.6. STOCK 30
3.3.6.1.CÓDIGO 31
3.3.6.2.TESTBENCH 35
3.3.7. CTR_Display 35
3.3.7.1.CÓDIGO 36
3.3.7.2.TESTBENCH 41
3.3.8. Multiplex 41
3.3.8.1.CÓDIGO 42
3.3.8.2.TESTBENCH 44
3.3.9. TOP 44
3.3.9.1.CÓDIGO 44
4. Github 52

5. Conclusión 52

1. Introducción y objetivos

1.1. Tarea a realizar

Como trabajo de la asignatura de Sistemas Electrónicos Digitales hemos decidido hacer una máquina expendedora en una placa NEXYS A7. El trabajo elegido era una de las propuestas presentadas por los profesores de la asignatura, dado que al no tener experiencia previa en el manejo de las FPGA's hemos preferido trabajar con lo que nos recomendaban. El enunciado del trabajo es el siguiente:

"Diseñe una máquina expendedora de refrescos. Admite monedas de 10c, 20c, 50c y 1€. Sólo admite el importe exacto, de forma que si introducimos dinero de más da un error y "devuelve" todo el dinero. Cuando se llega al importe exacto del refresco (1€) se activará una señal para dar el producto. Como entradas tendrá señales indicadoras de la moneda, señales indicadoras de producto y como salidas la señal de error y la de producto".

1.2. Objetivos

El principal propósito del trabajo es el diseño de la máquina expendedora como se propone en el enunciado, para llevar esto a cabo los principales objetivos son:

- Seleccionar entre varios productos.
- Sincronizar entradas asíncronas.
- Detectar flancos y generar pulsos.
- Mostrar por el Display información sobre el pago.
- Contar las monedas introducidas.
- Gestionar los errores.

Nosotros como objetivo extra hemos decidido añadir:

- Gestión de stock.
- Mayor cantidad de productos a poder elegir cada uno con su stock individual.
- Mostrar por pantalla toda la información real que tendría una máquina expendedora auténtica

2. Descripción del proyecto

2.1. Funcionamiento

Se utilizarán los códigos de las prácticas de la asignatura, que servirán de base para desarrollar parte del diseño y de la estructura del proyecto, la cual servirá como base para el resto del código.

Por lo tanto, los aspectos relacionados con el diseño de máquinas de estado, la sincronización, la detección de flancos y el manejo de los displays seguirán una estructura similar a la presentada en estas prácticas.

El funcionamiento de la máquina expendedora se describe de la siguiente manera: se comienza en un estado inicial en el que se debe seleccionar el producto deseado. Esto se hará con los 9 últimos switches de la FPGA, siendo cada switch un producto distinto, el cual se mostrará en el display la identificación del producto, y con los leds el stock de cada producto. Una vez seleccionado un producto se procederá a pulsar el botón de confirmación para pasar al estado de pago. En caso de que no hubiese stock de ese producto dará un mensaje de error y volverá de nuevo al estado de selección de producto.

Una vez en el estado de pago se mostrará en el display el precio del producto y las monedas faltantes para abonar el pago. Con cada botón que se pulse se irá restando dicha cantidad al pago inicial y cuando se llegue a 0 se deberá presionar de nuevo el botón de confirmar para que procese el pago.

Por último una vez realizado el pago se pasará al estado de expender en el cual se mostrará en el display un mensaje de que se está expendiendo el producto deseado. Tras un tiempo la máquina volverá de nuevo al estado inicial para poder elegir un nuevo producto.

La máquina estará diseñada de tal forma que no se interpretará si se pulsan 2 botones simultáneos y no permitirá la selección de dos productos al mismo tiempo. Evitando así fallos en la lógica y en el posterior funcionamiento de la máquina.

En todo momento se podrá realizar un reset general, el cual reseteará todos los parámetros que se hayan modificado durante el funcionamiento natural de la máquina, incluyendo el del stock. De forma que si nos hubiéramos quedado sin stock de varios productos podríamos realizar el reset para volver a rellenarlos. Esto simularía a un operario abriendo y rellenando una máquina expendedora real.

2.2. Entradas, salidas y constraints

2.2.1. Entradas

Puerto	Dirección	Tipo
`button_confir`	in	`STD_LOGIC`
`vector_botones_monedas`	in	`std_logic_vector`
`digsel`	in	`STD_LOGIC_VECTOR`
`CLK`	in	`std_logic`
`RST`	in	`STD_LOGIC`
`ESTADO_OFF_button`	in	`STD_LOGIC`
`control_LEDS_1`	in	`STD_LOGIC`
`control_LEDS_2`	in	`STD_LOGIC`

2.2.2. Salidas

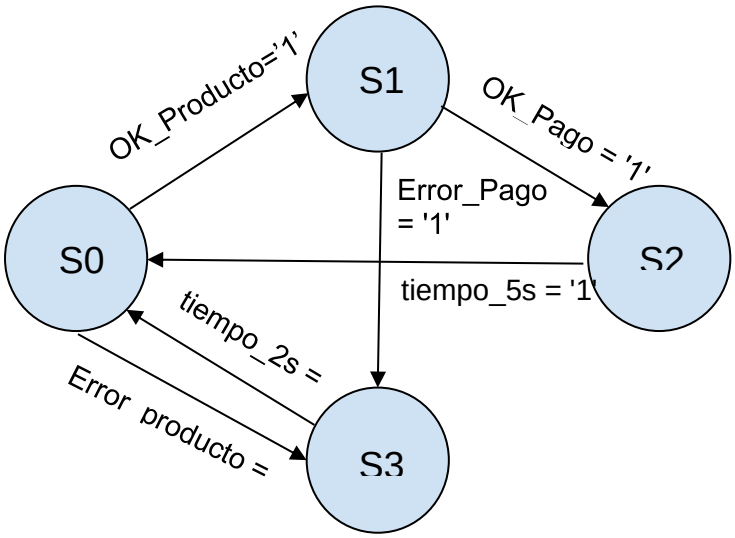
Puerto	Dirección	Tipo
`digselec`	out	`std_logic_vector`

Puerto	Dirección	Tipo
`segment`	out	`STD_LOGIC_VECTOR`
`Err`	out	`STD_LOGIC_VECTOR`

2.2.3. Constraints

Señal	Pin FPGA	Descripción
`CLK`	E3	Reloj 100MHz
`digsel[0]` a `digsel[8]`	J15, L16, M13, R15, R17, T18, U18, R13, T8	Interruptores de selección de producto (SW)
`ESTADO_OFF_button`	V10	Switch de Estado OFF (SW15)
`Err[0]` a `Err[15]`	H17 a V11	LEDs de depuración
`segment[0]` a `segment[7]`	T10 a H15	Segmentos del Display 7 Segmentos
`digselec[0]` a `digselec[7]`	J17 a U13	Anodos de selección de dígito del Display
`RST`	C12	Botón de Reset (CPU Resetn)
`button_confir`	N17	Botón de Confirmación (btnc)
`vector_botones_monedas [0]` a `[3]`	M18, P17, M17, P18	Botones para monedas (btnu, btnl, btnr, btnd)

2.3. Diagrama de estados

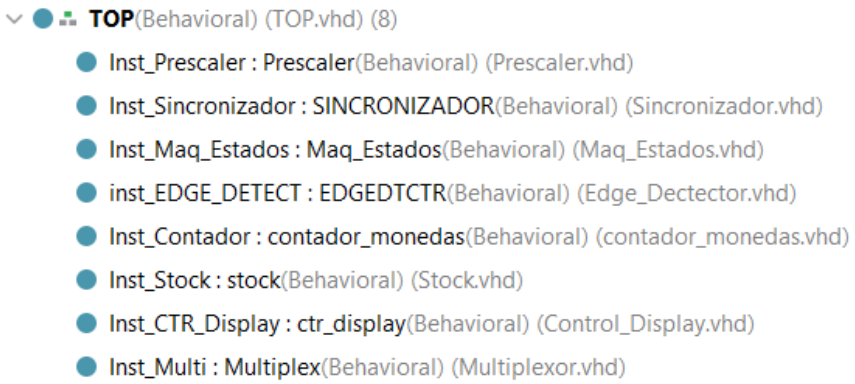


3. Desarrollo de los componentes del código y simulación

3.1. Relación y comunicación entre los bloques

Una vez definidos los objetivos y los requisitos que debe cumplir el proyecto, se procederá a planificar la estructura que dará forma al programa, con el fin de garantizar una correcta implementación de todas sus funcionalidades.

El diseño del sistema se llevará a cabo de forma que integre distintas entidades que interactuarán de manera coordinada, intercambiando información mediante variables auxiliares que conectarán sus entradas y salidas. Asimismo, cada entidad será responsable de ejecutar un conjunto específico de tareas, las cuales se describirán más adelante.



En primer lugar, encontramos la entidad del **Prescaler**, la cual a partir de la frecuencia interna de la FPGA logra sacar una frecuencia inferior para poder utilizar el bloque de **Multiplex** de forma segura, evitando que haya errores en el display.

A continuación las entidades **Sincronizador** y **EDGE_DETECT**, que, tal como hemos observado en las prácticas, se ocupan respectivamente de sincronizar las entradas utilizando con reloj interno de la FPGA y de identificar los cambios de estado por flancos. Su utilización resulta fundamental para minimizar los errores durante el proceso de lectura.

Por otro lado, contamos con tres entidades principales que estructuran el programa: **Maq_Estados**, **Contador** y **STOCK**.

Maq_Estados es una entidad que se encarga de controlar y sincronizar el resto de bloques utilizando una máquina de estados de Moore. La entidad de **Contador** se encarga de la contabilización de monedas, recibiendo las distintas señales de los botones y realizando las restas pertinentes al precio inicial del producto. **STOCK** es de las entidades más complejas, en ella se define un stock independiente para cada producto y se encarga de contabilizar el stock restante con cada nuevo pedido.

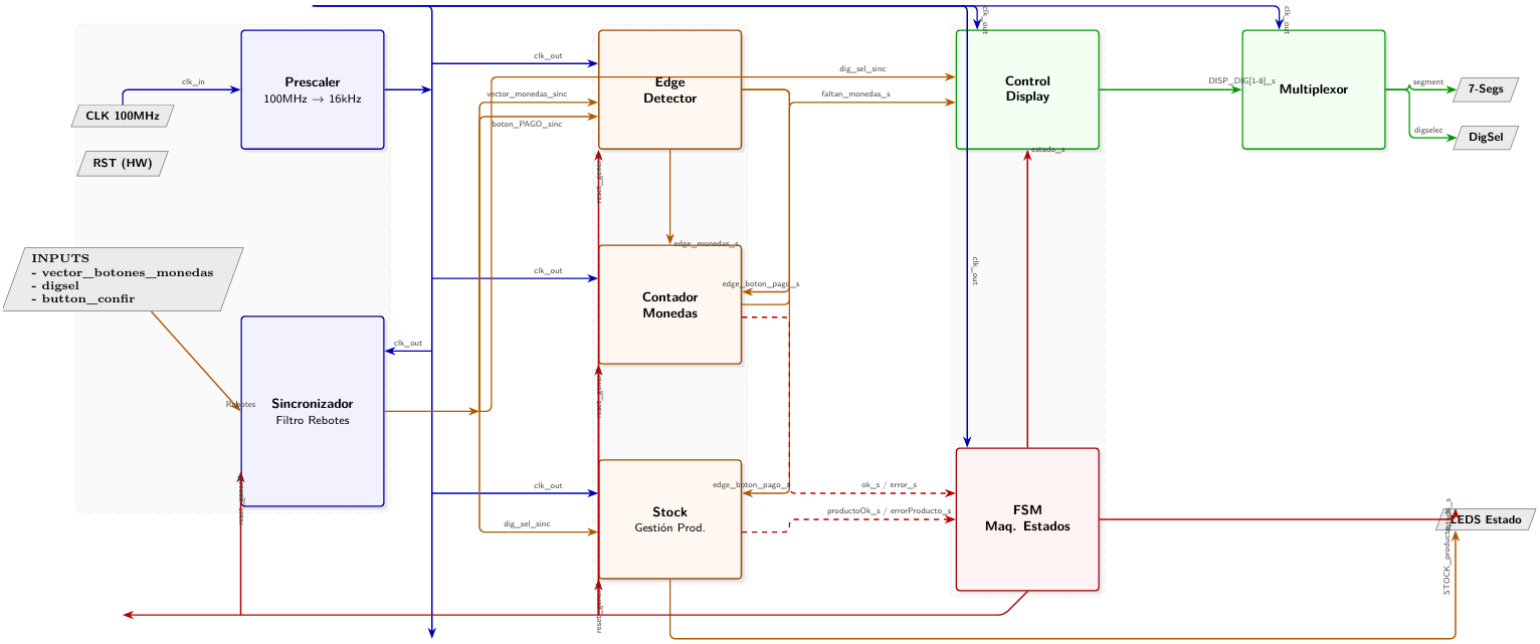
Acto seguido, para poder mostrar todo por el display se utilizarán las entidades de **CTR_Display** y de **Multiplex.**

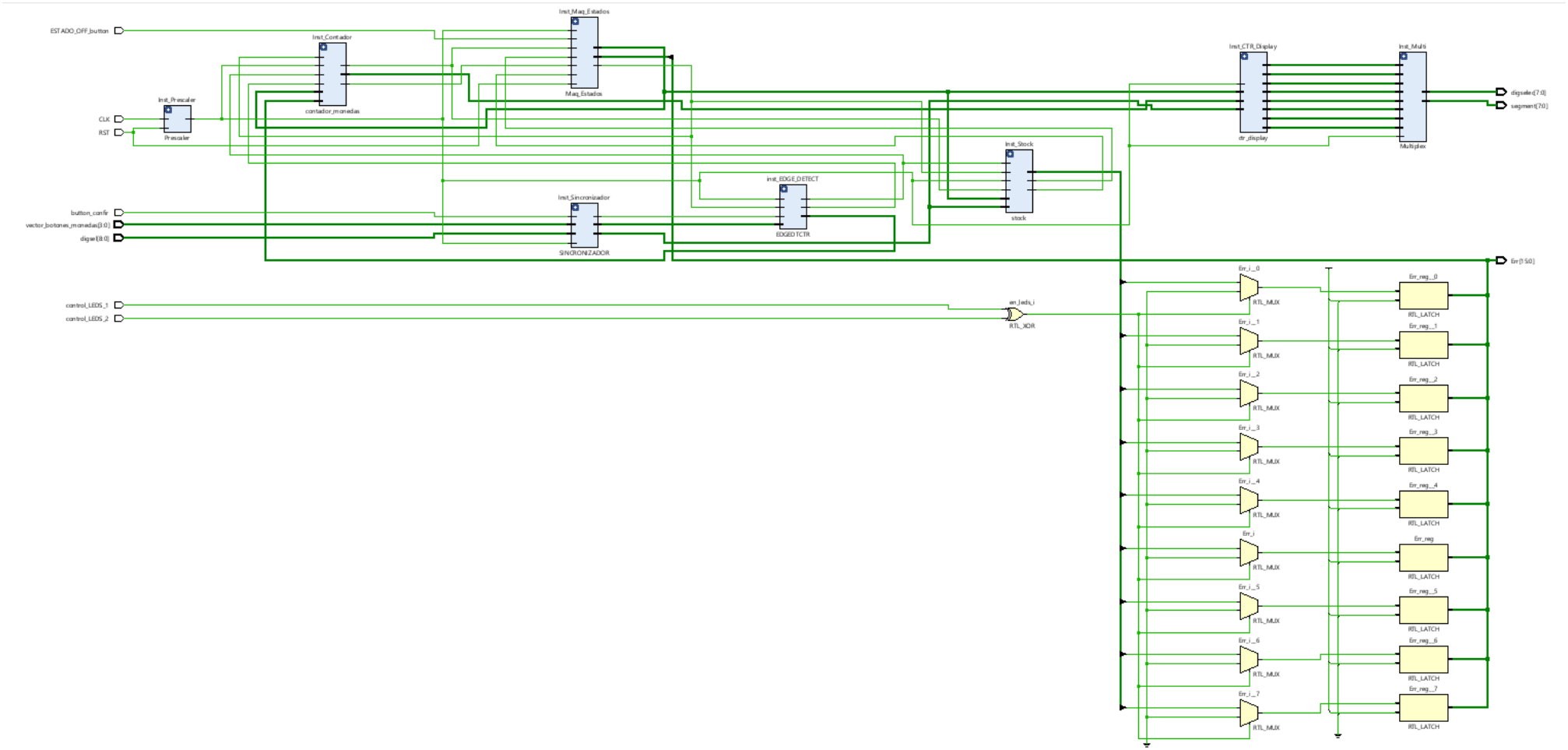
La entidad de **CTR_Display** es la que cuenta con un grado de complejidad mayor de las dos. Esta se encarga de seleccionar los segmentos de dentro del display que se deben de activar dependiendo del estado en el que se encuentre y de las acciones que se hayan tomado en cada estado.

La entidad de **Multiplex** se encarga primero de cargar de cargar la información que le llegue del **CTR_Display** en cada dígito del display. Y luego por cada pulso de reloj va encendiendo uno de los bloques y apagando el resto, avanzando el bloque ilumina con cada pulso hasta que vuelva a comenzar desde el principio.

Por último, contamos con una entidad superior denominada **TOP**, la cual no aparece reflejada en los diagramas pero cuya función es la más importante de todas. Esta entidad se dedica a declarar el resto de entidades y de llamarlas para poder poner en funcionamiento la máquina expendedora.

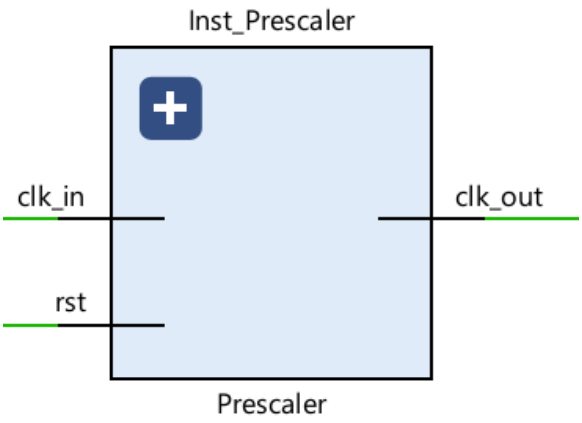
3.2. Esquema de conexión




ESTADO_OFF_button
CLK
RST
Inst_Prescaler
Prescaler
Inst_Contador
contador_monedas
Inst_Maq_Estados
Maq_Estados
button_confir
vector_botones_monedas[3:0]
digsel[8:0]
Inst_Sincronizador
SINCRONIZADOR
inst_EDGE_DETECT
EDGEDTCTR
Inst_Stock
stock
Inst_CTR_Display
ctr_display
Inst_Multi
Multiplex
digselec[7:0]
segment[7:0]
Err[15:0]
control_LEDS_1
control_LEDS_2
en_leds_i
RTL_XOR
Err_i_0
Err_i_1
Err_i_2
Err_i_3
Err_i_4
Err_i
Err_i_5
Err_i_6
Err_i_7
RTL_MUX
Err_reg_0
Err_reg_1
Err_reg_2
Err_reg_3
Err_reg_4
Err_reg
Err_reg_5
Err_reg_6
Err_reg_7
RTL_LATCH

3.3. Bloques

3.3.1. Prescaler



La función que tiene el bloque **PRESCALER** es la de recibir la frecuencia del reloj principal y transformar esta a una frecuencia inferior para que pueda ser compatible con el resto de componentes del sistema. En nuestro caso el reloj principal trabaja a 100 MHz y lo que buscamos es dividir esta frecuencia para obtener un reloj de 16 KHz. Esto se debe hacer ya que sino no podríamos ver las pantallas de los displays, ya que esto se haría a una velocidad imperceptible.

El funcionamiento del Prescaler funciona como un contador, es decir, en cada flanco ascendente de CLK_IN se va incrementando una unidad hasta llegar a un valor predefinido. Cuando la cuenta llega a este valor final se este se reinicia y la señal de salida recibe un pulso, consiguiendo así la nueva señal CLK_OUT con la frecuencia que se requiere.

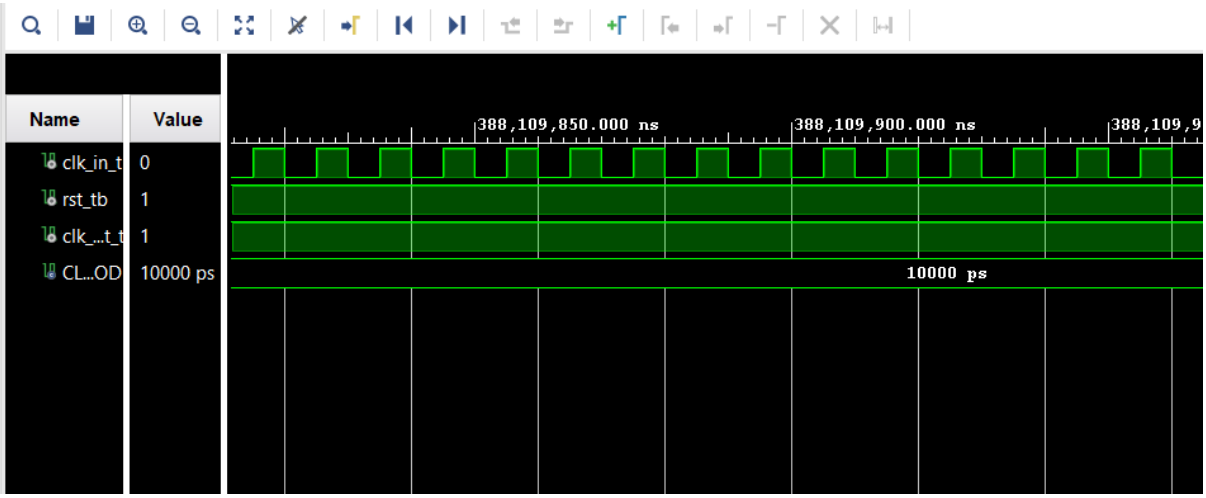
Por último, el Prescaler también dispone de un reset asíncrono activo a nivel bajo, el cual en caso de activarse inicializa el contador y pone la salida a cero. Cuando el reset vuelve a nivel alto, reiniciando el contador y haciendo que la señal de salida vuelva a oscilar. La constante *DIV_COUNT* es la que se usa para ajustar la frecuencia, por tanto, ajustando dicho valor se puede adaptar a otros diseños que requieran otras frecuencias.

3.3.1.1. CÓDIGO

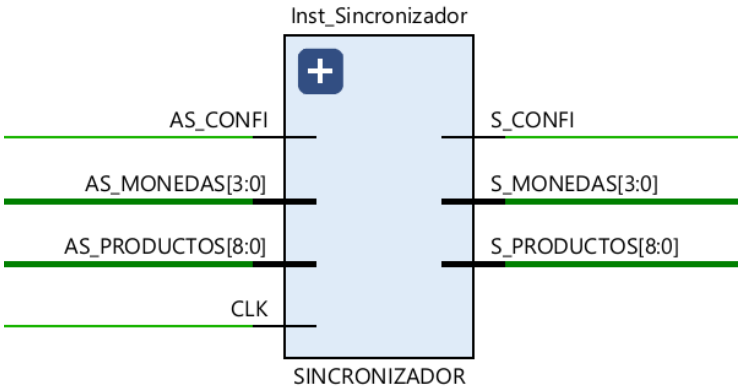
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity Prescaler is
    port(
```

```
        clk_in  : in  std_logic;   -- 100 MHz
        rst     : in  std_logic;   -- Reset síncrono
        clk_out : out std_logic    -- 16 kHz
    );
end Prescaler;

architecture Behavioral of Prescaler is

    constant DIV_COUNT : integer := 3124;  -- (100M/16k)/2 - 1
    signal counter     : integer range 0 to DIV_COUNT := 0;
    signal clk_reg     : std_logic := '0';

begin

    process(clk_in)
    begin
        if rising_edge(clk_in) then
            if rst = '0' then
                counter <= 0;
                clk_reg <= '0';
            else
                if counter = DIV_COUNT then
                    counter <= 0;
                    clk_reg <= not clk_reg; -- toggle del divisor
                else
                    counter <= counter + 1;
                end if;
            end if;
        end if;
    end process;

    clk_out <= clk_reg;

end Behavioral;
```

3.3.1.2. TESTBENCH



3.3.2. Sincronizador



El bloque **SINCRONIZADOR** es crucial para un correcto funcionamiento del proyecto. Tanto las entradas de la placa como los interruptores y los pulsadores son señales asíncronas. Si estas señales se pasarán tal cual podría causar problemas de metaestabilidad, esto consiste en que una señal se podría quedar en un estado indefinido entre '0' y '1' por un tiempo indeterminado, haciendo que el comportamiento de la máquina sea impredecible. Según la teoría, se recomienda pasar las señales que vienen del exterior por dos flip-flops, es decir, sincronizar en dos etapas para así eliminar la mayoría de los problemas y obtener una salida estable.

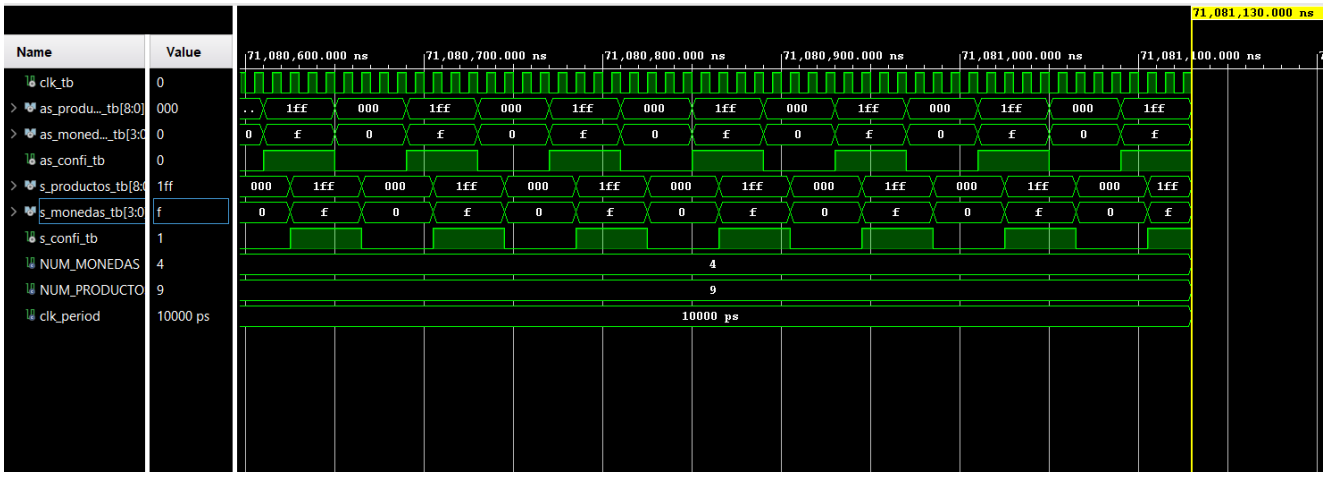
Para ello se han declarado dos registros (REG1_* y REG2_*) para cada una de las vectores asíncronos entrantes que se tienen, uno para las monedas y otro para los productos: AS_MONEDAS[3:0], AS_PRODUCTOS[8:0]. La entrada asíncrona del botón de confirmación se denomina AS_CONFI. En cada flanco ascendente del reloj de salida del **PRESCALER (**CLK_OUT ->CLK**)** el registro de la primera etapa coge el valor de entrada y se lo pasa al registro de la segunda etapa. Posteriormente los registros de esta segunda etapa se pasan a los registros finales de salida denominados: `S_MONEDAS[3:0]`, `S_PRODUCTOS[8:0]` y `S_CONFI`. De esta forma se consigue que las entradas exteriores que al principio eran asíncronos se alineen con el reloj.

3.3.2.1. CÓDIGO

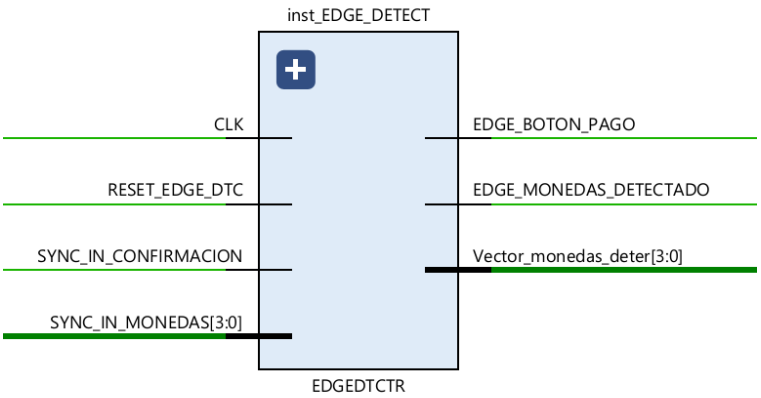
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity SINCRONIZADOR is
    generic(
        NUM_MONEDAS: positive:= 4;
        NUM_PRODUCTOS: positive:= 9
    );

    port(
```

```
        CLK: in std_logic;
        -- ENTRADAS ASINCRONAS DE LOS BOTONES Y SWITCHES
        AS_PRODUCTOS: in std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
        AS_MONEDAS: in std_logic_vector (NUM_MONEDAS - 1 downto 0);
        AS_CONFI : in std_logic;
        -- SALIDAS SINCRONAS DE LOS BOTONES Y SWITCHES
        S_PRODUCTOS: out std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
        S_MONEDAS: out std_logic_vector (NUM_MONEDAS - 1 downto 0);
        S_CONFI : out std_logic

    );


end SINCRONIZADOR;

architecture Behavioral of SINCRONIZADOR is

    signal REG1_PRODUCTOS: std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
    signal REG1_MONEDAS: std_logic_vector(NUM_MONEDAS - 1 downto 0);
    signal REG1_CONFI: std_logic;

    signal REG2_PRODUCTOS: std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
    signal REG2_MONEDAS: std_logic_vector(NUM_MONEDAS - 1 downto 0);
    signal REG2_CONFI: std_logic;

begin

    REGISTRO_1: process(CLK)
    begin
        if rising_edge(CLK) then
            REG1_PRODUCTOS<= AS_PRODUCTOS;
            REG1_MONEDAS<= AS_MONEDAS;
            REG1_CONFI <= AS_CONFI;

        end if;
    end process;

    REGISTRO_2:process(CLK)
    begin
        if rising_edge(CLK) then
            REG2_PRODUCTOS<= REG1_PRODUCTOS;
            REG2_MONEDAS<= REG1_MONEDAS;
            REG2_CONFI <= REG1_CONFI;
        end if;
    end process;

    S_MONEDAS <= REG2_MONEDAS;
    S_PRODUCTOS<= REG2_PRODUCTOS;
    S_CONFI <= REG2_CONFI;

end Behavioral;
```

3.3.2.2.TESTBENCH



3.3.3. EDGE_DETECT



El detector de flancos cumple dos funciones principales, por un lado detecta los cambios de estado en las señales de entrada y, por otro lado, genera pulsos de duración controlada para que puedan ser interpretados correctamente por el resto del sistema. Para ello, se compara el valor actual de la señal con su valor anterior, lo que permite identificar flancos ascendentes o descendentes.

Cada uno de los cuatro botones de moneda (SYNC_IN_MONEDAS[3:0]) se gestiona mediante un registro de desplazamiento de tres bits, que almacena la señal de entrada sincronizada en cada ciclo de reloj (CLK). Cuando en alguno de estos registros se detecta un flanco ascendente se marca la moneda correspondiente como candidata. A continuación, se verifica que solo se haya activado una moneda, si se detectan varias a la vez, no se genera ningún pulso en `EDGE_MONEDAS_DETECTADO`. Si la detección es válida, se activa un pulso de duración fija mediante un contador cargado con el valor *NUM_PULSOS*, y se registra de forma determinista qué moneda ha sido introducida (`Vector_modedas_deter[3:0]`). Este pulso prolongado permite eliminar los efectos de los rebotes mecánicos y asegura que cada moneda se contabilice una única vez.

De forma similar, se trabaja con el botón de confirmación(SYNC_IN_CONFIRMACION). Cuando se detecta un flanco ascendente en su registro, se genera un pulso con una duración controlada en EDGE_BOTON_PAGO, independientemente de cuánto tiempo permanezca pulsado el botón. Además, al confirmarse el pago, se anulan los pulsos de moneda pendientes y se borra el registro de monedas, evitando que se sigan acumulando valores después de la confirmación.

Gracias a este detector de flancos, los módulos posteriores reciben señales limpias y de duración constante que indican de manera fiable la introducción de monedas o la confirmación del pago. Esto evita conteos múltiples no deseados y simplifica el diseño del sistema de control.

3.3.3.1.CÓDIGO

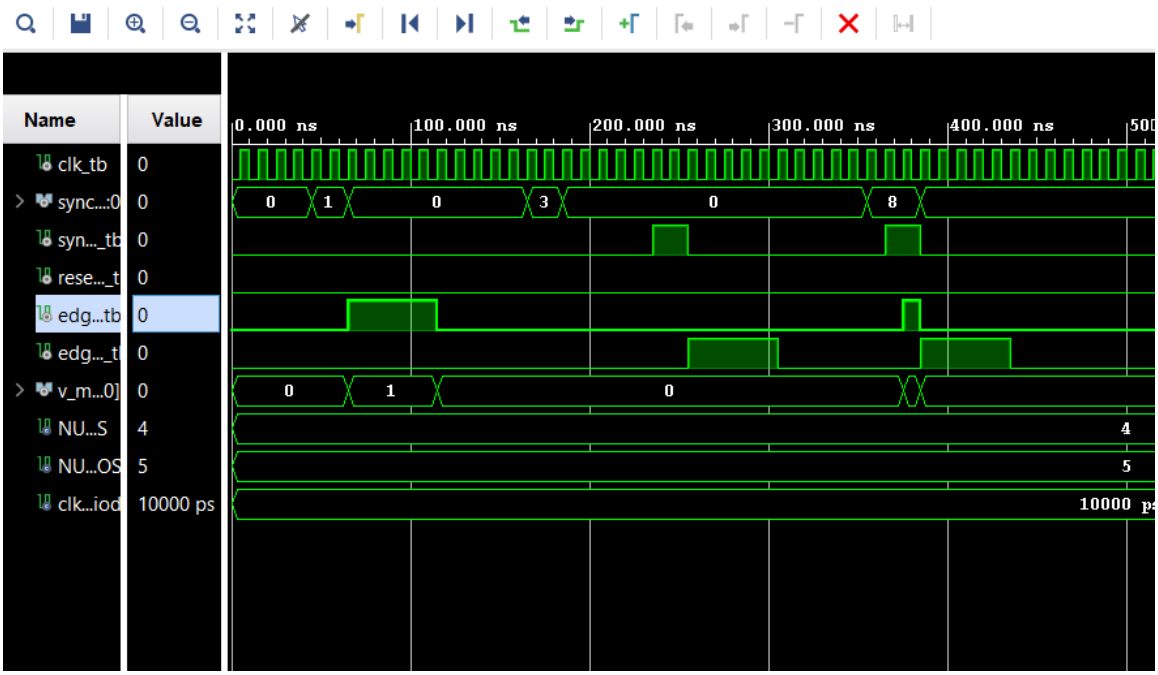
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity EDGEDTCTR is
    Generic(
            NUM_MONEDAS: positive:= 4;
            NUM_PULSOS: positive := 5
    );

    Port (
            CLK : in STD_LOGIC;

           -- BOTONES QUE REQUIEREN DE UN FLANCO para definir un estado de activación

           -- EMTRADAS DE LOS BOTONES
           SYNC_IN_MONEDAS : in STD_LOGIC_VECTOR (NUM_MONEDAS -1 downto 0);
           SYNC_IN_CONFIRMACION: in STD_LOGIC;
           RESET_EDGE_DTC : in STD_LOGIC;

           -- SALIDAS como FLANCOS DE SUBIDA
           EDGE_MONEDAS_DETECTADO : out STD_LOGIC;
           EDGE_BOTON_PAGO : out STD_LOGIC;

           -- Vector para monedas, si se detecta que una de las 4 monedas se activa
sacamos un vector
           -- determinista que define qué moneda ha entrado y lo mantiene encendido hasta
           Vector_monedas_deter: out STD_LOGIC_VECTOR (NUM_MONEDAS -1 downto 0)
           );
end EDGEDTCTR;


architecture Behavioral of EDGEDTCTR is

    -- DETECCION DE FLANCO CON VECTORES DE 3 BITS ( van desplazandose temporalmente)
    -- Registros para los botones de monedas ( 10, 20 , 50 y 1€)
    signal sreg_1: std_logic_vector (2 downto 0) := (others => '0');
```

```
    signal sreg_2: std_logic_vector (2 downto 0) := (others => '0');
    signal sreg_3: std_logic_vector (2 downto 0) := (others => '0');
    signal sreg_4: std_logic_vector (2 downto 0) := (others => '0');
    -- registro del boton de confirmacion de pago (renombrado como pediste)
    signal sreg_boton_pago: std_logic_vector (2 downto 0) := (others => '0');


    -- vector candidato a ser el determinista (el que queda permanente para avisar al
contador de qué moneda es)
    -- Esto del determinismo se hace por una cuestion de seguridad, NO QUEREMOS 2 monedas
al mismo tiempo
    signal Vect_MONEDAS_POSIBLE_s : std_logic_vector (NUM_MONEDAS -1 downto 0) := (others
=> '0');
    signal Vect_MONEDAS_DETERMINISTA : std_logic_vector (NUM_MONEDAS -1 downto 0) :=
(others => '0');


    -- FLANCOS DE 5 CLOCKS DE ACTIVACION
    -- Registros para el Flanco de MONEDAS que dura 5 ciclos
    signal r_edge_monedas : std_logic := '0';
    signal pulsos_flanco_monedas  : unsigned(NUM_PULSOS-1 downto 0) := (others => '0');
-- hasta 5
    -- Registros para el Flanco DEL BOTON CONFIRMAR PAGO que dura 5 ciclos
    signal r_edge_boton_pago : std_logic := '0';
    signal pulsos_flanco_boton  : unsigned(NUM_PULSOS-1 downto 0) := (others => '0');  --
hasta 5





begin
    process (CLK)
    begin
        if rising_edge(CLK) then
            sreg_1 <= sreg_1(1 downto 0) & SYNC_IN_MONEDAS(0);
            sreg_2 <= sreg_2(1 downto 0) & SYNC_IN_MONEDAS(1);
            sreg_3 <= sreg_3(1 downto 0) & SYNC_IN_MONEDAS(2);
            sreg_4 <= sreg_4(1 downto 0) & SYNC_IN_MONEDAS(3);
       ------------------------------------------------------------------------------
EDGE DETECTOR DE LA PRÁCTICA 2 PARA EL BOTÓN DE CONFIRMAR
            sreg_boton_pago <= sreg_boton_pago(1 downto 0) & SYNC_IN_CONFIRMACION;

        end if;
    end process;


    ---------------------------------------------------------------------------
    -- DETECCION DE FLANCO (combinacional, fuera del clk)
    ---------------------------------------------------------------------------

    with sreg_1 select
        Vect_MONEDAS_POSIBLE_s(0) <= '1' when "001", '0' when others;
    with sreg_2 select
        Vect_MONEDAS_POSIBLE_s(1) <= '1' when "001", '0' when others;
    with sreg_3 select
```

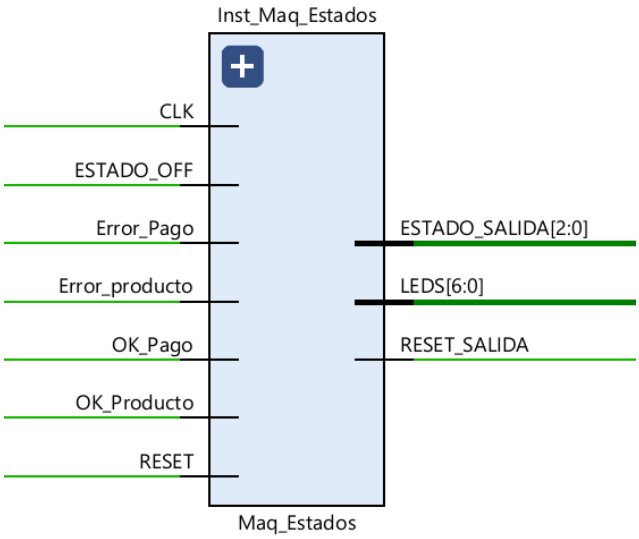
```
        Vect_MONEDAS_POSIBLE_s(2) <= '1' when "001", '0' when others;
    with sreg_4 select
        Vect_MONEDAS_POSIBLE_s(3) <= '1' when "001", '0' when others;




-- Pulso estirado 5 clocks para BOTÓN PAGO (depende de CLK para poder contar los 5 clks)
    process (CLK)
    begin
        if rising_edge(CLK) then

            if pulsos_flanco_boton = to_unsigned(0, NUM_PULSOS) then
                if sreg_boton_pago = "001" then
                    pulsos_flanco_boton <= to_unsigned(NUM_PULSOS, NUM_PULSOS);
                    r_edge_boton_pago   <= '1';
                else
                    r_edge_boton_pago <= '0';
                end if;

            else
                if pulsos_flanco_boton = to_unsigned(1, NUM_PULSOS) then
                    pulsos_flanco_boton <= (others => '0');
                    r_edge_boton_pago   <= '0';
                else
                    pulsos_flanco_boton <= pulsos_flanco_boton - 1;
-- RESTA
                    r_edge_boton_pago   <= '1';
                end if;
            end if;

        end if;
    end process;


    -- Pulso estirado 5 clocks para MONEDAS + vector determinista
    process (CLK)
    begin
        if rising_edge(CLK) then

            -- PAGO ANULA MONEDAS
            if (sreg_boton_pago = "001") or
               (pulsos_flanco_boton /= to_unsigned(0, NUM_PULSOS)) then

                pulsos_flanco_monedas     <= (others => '0');
                r_edge_monedas            <= '0';
                Vect_MONEDAS_DETERMINISTA <= (others => '0');

            else
                if pulsos_flanco_monedas = to_unsigned(0, NUM_PULSOS) then

                    if (Vect_MONEDAS_POSIBLE_s = "0001" or
                        Vect_MONEDAS_POSIBLE_s = "0010" or
                        Vect_MONEDAS_POSIBLE_s = "0100" or
                        Vect_MONEDAS_POSIBLE_s = "1000") then
```

```
                    pulsos_flanco_monedas     <= to_unsigned(NUM_PULSOS, NUM_PULSOS);
                    r_edge_monedas            <= '1';
                    Vect_MONEDAS_DETERMINISTA <= Vect_MONEDAS_POSIBLE_s;

                else
                    r_edge_monedas            <= '0';
                    Vect_MONEDAS_DETERMINISTA <= (others => '0');
                end if;

            else
                if pulsos_flanco_monedas = to_unsigned(1, NUM_PULSOS) then
                    pulsos_flanco_monedas     <= (others => '0');
                    r_edge_monedas            <= '0';
                    Vect_MONEDAS_DETERMINISTA <= (others => '0');
                else
                    pulsos_flanco_monedas <= pulsos_flanco_monedas - 1;
                    r_edge_monedas        <= '1';
                end if;
            end if;

        end if;

    end if;
end process;


-- Salidas desde registros (lo que se estira es el registro)
EDGE_MONEDAS_DETECTADO <= r_edge_monedas;
EDGE_BOTON_PAGO        <= r_edge_boton_pago;
Vector_monedas_deter   <= Vect_MONEDAS_DETERMINISTA;

end Behavioral;
```

3.3.3.2.TESTBENCH



3.3.4. Maq_Estados



La entidad **Maq_Estados** se encarga de controlar el funcionamiento general del sistema mediante una máquina de estados finitos. Para ello se define el tipo enumerado *ESTADOS*, que incluye los estados *SOFF*, *S0*, *S1*, *S2* y *S3*, representando las distintas fases del sistema. El estado actual se actualiza en el proceso *state_register* en cada flanco de reloj.

Las transiciones entre estados se definen en el proceso *Cambio_Estados*. Si la señal de entrada ESTADO_OFF pasa a cero, el sistema entra en el estado SOFF. Desde este estado, la máquina retorna automáticamente a *S0* cuando ESTADO_OFF vuelve a activarse. En el estado *S0*, la detección de un error en la selección del producto (Error_Producto) provoca el paso a *S3*, mientras que una selección correcta (OK_Producto) conduce al estado *S1*. Desde *S1* un error en el pago (Error_Pago) lleva de nuevo a *S3*, y un pago correcto (OK_Pago) permite avanzar al estado *S2*. En *S2* una vez transcurridos cinco segundos, el sistema regresa a *S0*. De manera similar, en el estado *S3* se vuelve a *S0* tras un retardo de dos segundos. De esta forma se cubre todo el ciclo de selección, pago, entrega del producto y gestión de errores.

La codificación de los estados para su visualización o control externo se realiza en el proceso *output_decod*, que asigna un valor binario a la salida `ESTADO_SALIDA` en función del estado activo. Así *S0* se representa como “000”, *S1* como “001”, *S2* como “010”, S3 como “011” y *SOFF* como “111”.

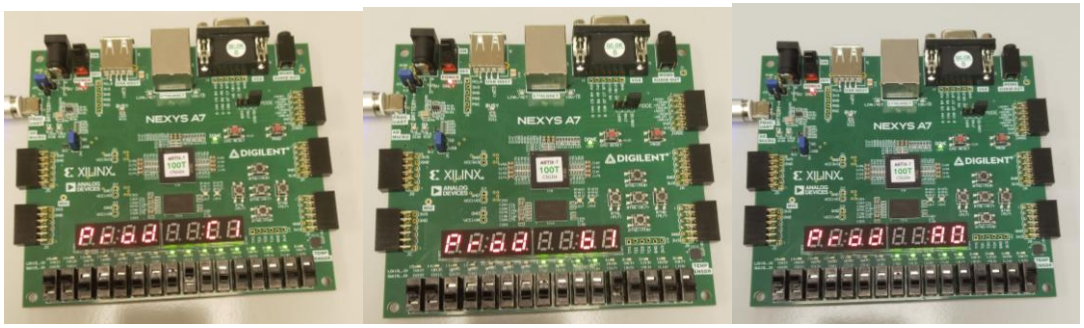
Por otro lado, el proceso *Reset_General* genera la señal `RESET_SALIDA`, que se desactiva cuando las entradas RESET o ESTADO_OFF están a cero y permanece activa en el resto de situaciones.

Los retardos temporales necesarios se implementan mediante dos temporizadores, estos generan las señales *tiempo_2s* y *tiempo_5s* respectivamente, permitiendo así las transiciones automáticas de retorno al estado inicial.

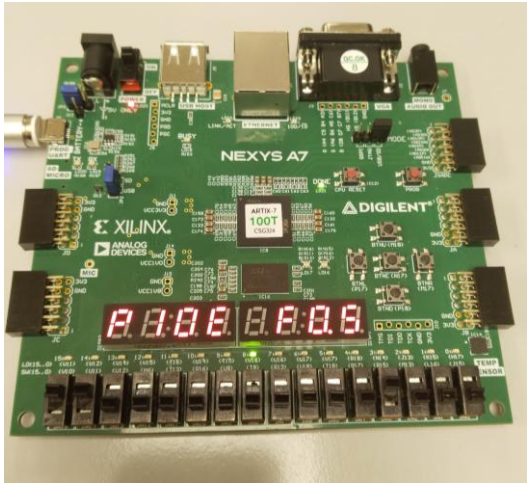
Por último, el proceso *CONTROL_LEDS*, se encarga de encender una secuencia de leds (LEDS) simulando que se está sacando el producto. Esto solo pasará cuando nos encontremos en el estado *S2*.

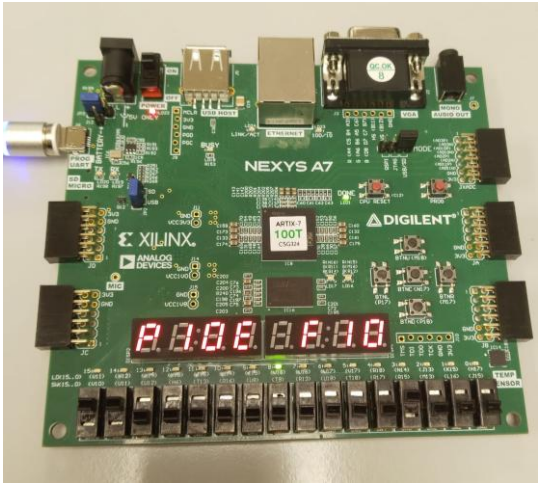
3.3.4.1. FOTOS DEL PROCESO

1. *SELECCIONANDO PRODUCTO*

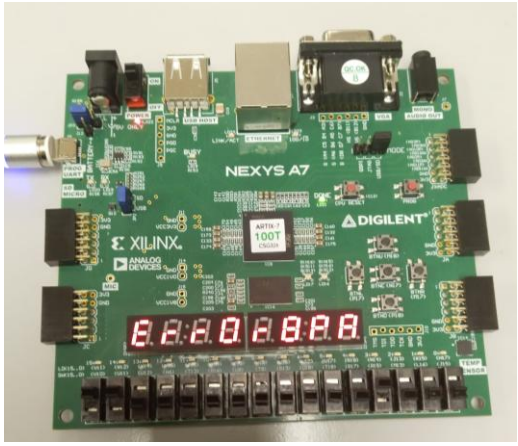


2. PAGANDO PRODUCTO

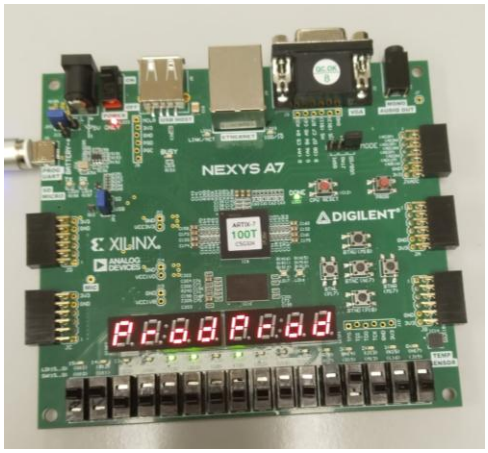




3. ESTADO ERROR



4. EXPENDIENDO PRODUCTO



3.3.4.2.CÓDIGO

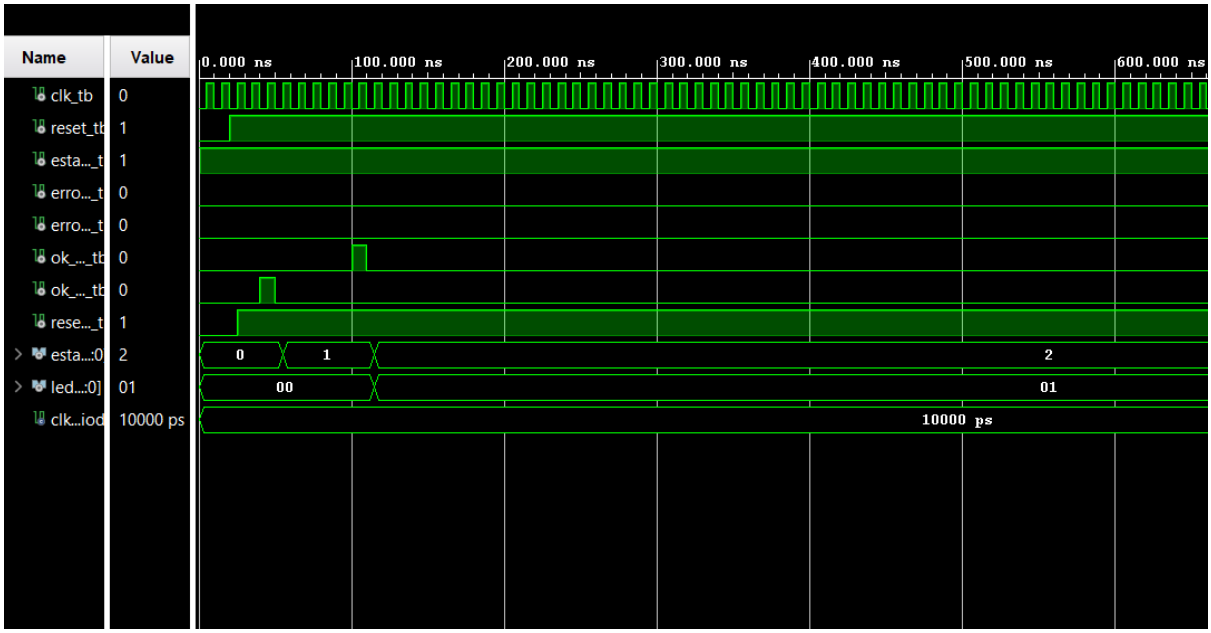
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Maq_Estados is
    Port(
            Error_producto      : in std_logic;
            Error_Pago          : in std_logic;
            OK_Pago             : in std_logic;
            OK_Producto         : in std_logic;
            RESET               : in std_logic;
            CLK                 : in std_logic;
            ESTADO_OFF          : in std_logic;

            RESET_SALIDA :  out std_logic;
            ESTADO_SALIDA : out unsigned(2 downto 0);

            LEDS : out STD_LOGIC_VECTOR (6 downto 0)

            );
end Maq_Estados;

architecture Behavioral of Maq_Estados is
   type ESTADOS is (S0, S1, S2, S3, SOFF);
   signal current_state: ESTADOS := SOFF;
   signal next_state: ESTADOS;

   signal tiempo_2s: std_logic := '0';
   signal tiempo_5s: std_logic := '0';

   signal contador_2s : integer range 0 to 32000 := 0;
   signal contador_5s : integer range 0 to 80000 := 0;

   signal counter : integer := 0;

begin

-- Registro de estado: RESET siempre a S0 (como antes)
state_register: process (RESET, CLK)
begin
    if RESET = '0' then
        current_state <= S0;
    elsif rising_edge(CLK) then
        current_state <= next_state;
    end if;
end process;

-- Lógica de transición (combinacional) SIN process(all)
Cambio_Estados: process (clk)
begin
    next_state <= current_state;

    if ESTADO_OFF = '0' then
        next_state <= SOFF;
```

```
    else
        case current_state is
            when SOFF => next_state <= S0;

            when S0 =>
                if Error_producto = '1' then
                    next_state <= S3;
                elsif OK_Producto = '1' then
                    next_state <= S1;
                end if;

            when S1 =>
                if Error_Pago = '1' then
                    next_state <= S3;
                elsif OK_Pago = '1' then
                    next_state <= S2;
                end if;

            when S2 =>
                if tiempo_5s = '1' then
                    next_state <= S0;
                end if;

            when S3 =>
                if tiempo_2s = '1' then
                    next_state <= S0;
                end if;

            when others =>
                next_state <= S0;
        end case;
    end if;
end process;

-- Decodificación de salida (combinacional)
output_decod: process (current_state)
begin
    case current_state is
        when S0 =>
            ESTADO_SALIDA <= to_unsigned(0, ESTADO_SALIDA'length);

        when S1 =>
            ESTADO_SALIDA <= to_unsigned(1, ESTADO_SALIDA'length);

        when S2 =>
            ESTADO_SALIDA <= to_unsigned(2, ESTADO_SALIDA'length);

        when S3 =>
            ESTADO_SALIDA <= to_unsigned(3, ESTADO_SALIDA'length);

        when SOFF =>
            ESTADO_SALIDA <= "111";

        when others =>
            ESTADO_SALIDA <= to_unsigned(0, ESTADO_SALIDA'length);
    end case;
```

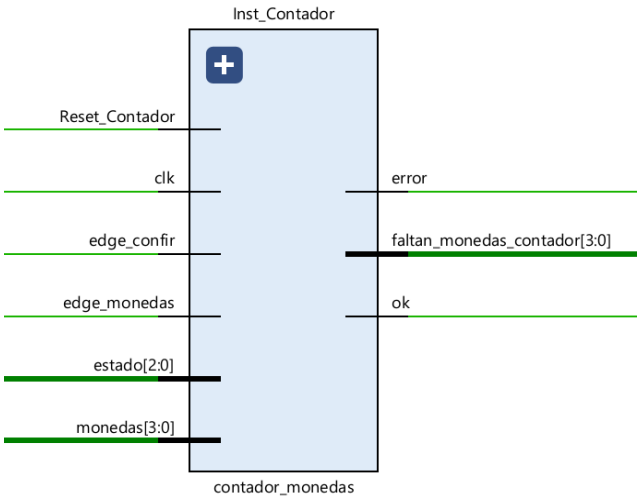
```
end process;

-- Reset general hacia el resto del sistema (activo a '0')
-- Si RESET='0' -> reseteas todo
-- Si ESTADO_OFF='0' -> también reseteas todo (apagado)
Reset_General: process (RESET, CLK)
begin
    if RESET = '0' then
        RESET_SALIDA <= '0';
    elsif rising_edge(CLK) then
        if ESTADO_OFF = '0' then
            RESET_SALIDA <= '0';
        else
            RESET_SALIDA <= '1';
        end if;
    end if;
end process;

-- Temporizador 2s (secuencial, solo clk; y rearmado por RESET/ESTADO_OFF/SOFF dentro)
Temporizador_de_2s: process (CLK)
begin
    if rising_edge(CLK) then
        if (RESET = '0') or (ESTADO_OFF = '0') or (current_state = SOFF) then
            contador_2s <= 0;
            tiempo_2s   <= '0';

        elsif current_state = S3 then
            if contador_2s < 32000 then
                contador_2s <= contador_2s + 1;
                tiempo_2s   <= '0';
            else
                contador_2s <= 32000;
                tiempo_2s   <= '1';
            end if;

        else
            contador_2s <= 0;
            tiempo_2s   <= '0';
        end if;
    end if;
end process;

-- Temporizador 5s (secuencial, solo clk; y rearmado por RESET/ESTADO_OFF/SOFF dentro)
Temporizador_de_5s: process (CLK)
begin
    if rising_edge(CLK) then
        if (RESET = '0') or (ESTADO_OFF = '0') or (current_state = SOFF) then
            contador_5s <= 0;
            tiempo_5s   <= '0';

        elsif current_state = S2 then
            if contador_5s < 80000 then
                contador_5s <= contador_5s + 1;
                tiempo_5s   <= '0';
            else
                contador_5s <= 80000;
```

```
                tiempo_5s   <= '1';
            end if;

        else
            contador_5s <= 0;
            tiempo_5s   <= '0';
        end if;
    end if;
end process;


Control_LEDS: process(current_state)
    begin

    case current_state is

            when S2 =>
                if contador_5s < 10000 then
                    LEDS(0)<='1';  -- Valor 1
                elsif contador_5s < 20000 then
                    LEDS(1)<='1';  -- Valor 2
                elsif contador_5s < 30000 then
                    LEDS(2)<='1';  -- Valor 3
                elsif contador_5s < 40000 then
                    LEDS(3)<='1';  -- Valor 4
                elsif contador_5s < 50000 then
                    LEDS(4)<='1';  -- Valor 2
                elsif contador_5s < 60000 then
                    LEDS(5)<='1';  -- Valor 3
                elsif contador_5s < 70000 then
                    LEDS(6)<='1';  -- Valor 4
                elsif contador_5s = 80000 then
                    counter <= 0;
                    LEDS(0)<='0';
                    LEDS(1)<='0';
                    LEDS(2)<='0';
                    LEDS(3)<='0';
                    LEDS(4)<='0';
                    LEDS(5)<='0';
                    LEDS(6)<='0';
                end if;

            when others =>
                    counter <= 0;
                    LEDS(0)<='0';
                    LEDS(1)<='0';
                    LEDS(2)<='0';
                    LEDS(3)<='0';
                    LEDS(4)<='0';
                    LEDS(5)<='0';
                    LEDS(6)<='0';
     end case;
  end process;

end Behavioral;
```

3.3.4.3.TESTBENCH



3.3.5. Contador



La entidad **Contador** se encarga de contabilizar las monedas que le entran a la máquina expendedora. De manera que en caso de que se haya introducido el número de monedas correcto y se confirme el pago se pase al siguiente estado. En caso de que no se haya introducido la cantidad necesaria para pagar el producto, si se pulsa el botón de confirmar se pasará a un estado de error.

Para implementar esta entidad, disponemos de la entrada estado[2:0], que es un vector de 3 elementos y representa los posibles estados en los que se puede encontrar la máquina. Mediante el vector de entrada monedas[3:0] se introduce el dinero en la máquina. Cada bit del vector monedas[3:0] representa un tipo de moneda: "0001" para 10 céntimos, "0010" para 20 céntimos, "0100" para 50 céntimos y "1000" para 1€. Por cada flanco de subida de CLK se actualizan las variables y los estados del código. En caso de que se encuentre activa la entrada Reset_Contador se realizará un reset de las variables de la entidad.

Como medida de seguridad para que no se puedan introducir varias monedas a la vez, solo se guardará en el registro de monedas la entrada monedas[3:0] si se encuentra activa ('1') la entrada edge_monedas.

A medida que se van introduciendo las monedas se van restando al valor final del producto y se mandará dicho valor restante por la salida `faltan_monedas_contador[3:0]`. En caso de que se pulse el botón de confirmación (edge_confir) y el pago sea correcto se activará la salida ok y se cambiará al estado final. En caso de que se pulse el botón y el pago sea incorrecto se activará la salida `error`.

3.3.5.1.CÓDIGO

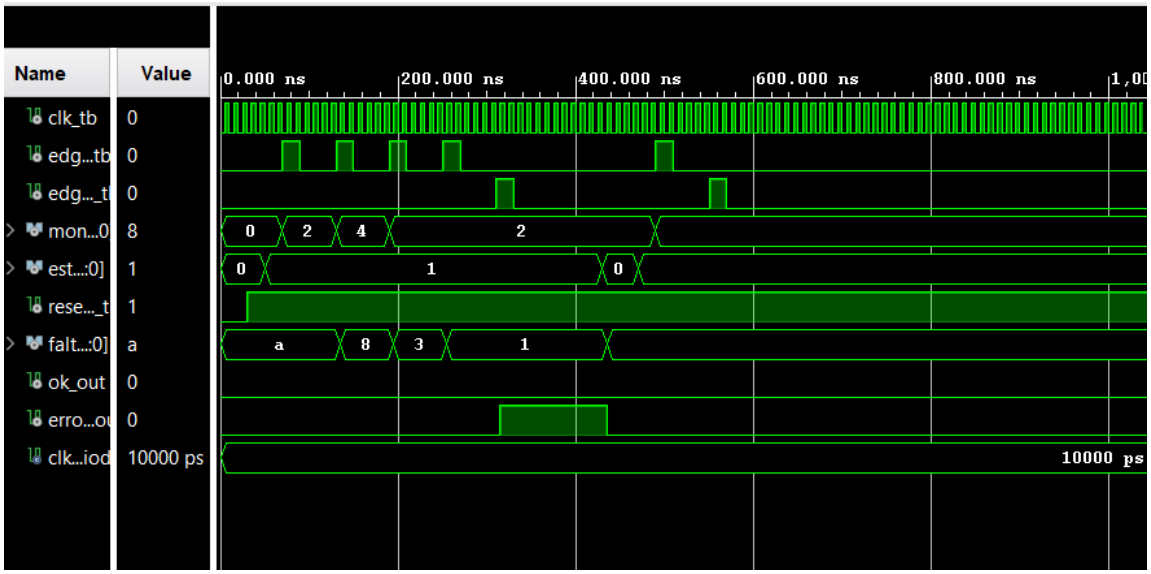
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity contador_monedas is
 port(
        clk            : in  std_logic;
        edge_monedas   : in  std_logic; -- flanco que indica que la moneda se ha metido
        edge_confir    : in  std_logic; -- boton de confirmar el pago cuando se hayan
metido todas las monedas
        monedas        : in  std_logic_vector(3 downto 0); -- vector que representa qué
moneda se ha metido (10, 20, 50, 1€)
        estado         : in  unsigned(2 downto 0); -- Entrada de la FSM que indica el
estado global
        Reset_Contador : in std_logic ;

        faltan_monedas_contador : out unsigned(3 downto 0); -- Número de monedas que
falta, de cara al DISPLAY para que pueda mostrarlo por pantalla
        ok             : out std_logic; -- Señal para la FSM de que el pago se ha podido
realizar correctamente
        error          : out std_logic -- Señal para la FSM de que el pago ha sido
erroneo, por pasarse o quedarse corto con el importe
    );
end contador_monedas;

architecture Behavioral of contador_monedas is

  constant ESTADO_0 : unsigned(2 downto 0) := (others => '0');
```

```
  constant ESTADO_1 : unsigned(2 downto 0) := "001";

  signal faltan_reg    : unsigned(3 downto 0) := to_unsigned(10, 4); -- 0..10
  signal resta         : unsigned(3 downto 0) := (others => '0');
  signal estado_prev   : unsigned(2 downto 0) := (others => '0');

  signal ok_pago       : std_logic := '0';
  signal error_pago    : std_logic := '0';
  signal pago_excedido : std_logic := '0';

  -- AÑADIDO (mínimo): detectar evento 1 vez aunque edge_* dure varios clocks
  signal edge_monedas_prev : std_logic := '0';
  signal edge_confir_prev  : std_logic := '0';
  signal monedas_guardadas : std_logic_vector(3 downto 0) := (others => '0');

begin

  process(clk)
    variable resta_calc : unsigned(3 downto 0);
    variable pulso_monedas : std_logic;
    variable pulso_confir  : std_logic;
  begin
    if rising_edge(clk) then

      resta_calc := (others => '0');

      -- AÑADIDO: pulso interno de 1 ciclo
      pulso_monedas := '0';
      pulso_confir  := '0';

      if (edge_monedas = '1') and (edge_monedas_prev = '0') then
        pulso_monedas := '1';
      end if;

      if (edge_confir = '1') and (edge_confir_prev = '0') then
        pulso_confir := '1';
      end if;

      if pulso_monedas = '1' then
        monedas_guardadas <= monedas;
      end if;

      ----------------------------------------------------------------
      -- RESET GLOBAL (funciona en cualquier estado)
      ----------------------------------------------------------------
      if Reset_Contador = '0' then
        faltan_reg    <= to_unsigned(10, 4);
        resta         <= (others => '0');
        pago_excedido <= '0';
        ok_pago       <= '0';
        error_pago    <= '0';
        monedas_guardadas <= (others => '0');

      else
        ----------------------------------------------------------------
        -- Al volver a ESTADO_0: reinicio total
```

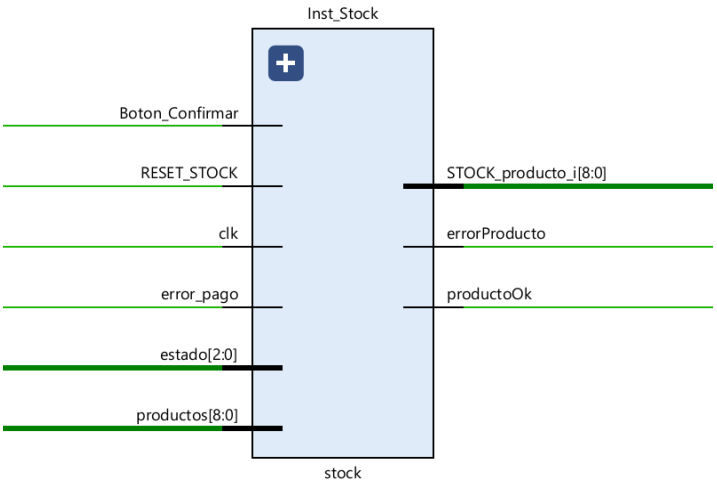
```
      ---------------------------------------------------------------
      if (estado_prev /= ESTADO_0) and (estado = ESTADO_0) then
        faltan_reg    <= to_unsigned(10, 4);
        resta         <= (others => '0');
        pago_excedido <= '0';
        ok_pago       <= '0';
        error_pago    <= '0';
        monedas_guardadas <= (others => '0');
      end if;

      ---------------------------------------------------------------
      -- Si salimos de ESTADO_1, borramos el contexto de pago
      ---------------------------------------------------------------
      if (estado_prev = ESTADO_1) and (estado /= ESTADO_1) then
        faltan_reg    <= to_unsigned(10, 4);
        resta         <= (others => '0');
        pago_excedido <= '0';
        monedas_guardadas <= (others => '0');
      end if;

      ---------------------------------------------------------------
      -- OPERACIÓN SOLO EN ESTADO_1
      ---------------------------------------------------------------
      if (estado = ESTADO_1) then

        -- Si ya hay resultado, no aceptar más acciones
        if (ok_pago = '0') and (error_pago = '0') then

          ---------------------------------------------------------
          -- Inserción de moneda
          ---------------------------------------------------------
          if pulso_monedas = '1' then

            -- monedas: one-hot
            -- (0)=10c, (1)=20c, (2)=50c, (3)=1€
            if monedas_guardadas(0) = '1' then
              resta_calc := to_unsigned(1, 4);
            elsif monedas_guardadas(1) = '1' then
              resta_calc := to_unsigned(2, 4);
            elsif monedas_guardadas(2) = '1' then
              resta_calc := to_unsigned(5, 4);
            elsif monedas_guardadas(3) = '1' then
              resta_calc := to_unsigned(10, 4);
            else
              resta_calc := (others => '0');
            end if;

            resta <= resta_calc;

            if resta_calc /= to_unsigned(0, 4) then
              if faltan_reg > resta_calc then
                faltan_reg <= faltan_reg - resta_calc;
              elsif faltan_reg = resta_calc then
                faltan_reg <= (others => '0');
              else
                faltan_reg    <= (others => '0');
```

```
                pago_excedido <= '1';
              end if;
            end if;

          ------------------------------------------------------------
          -- Confirmar pago
          ------------------------------------------------------------
          elsif pulso_confir = '1' then

            if faltan_reg = to_unsigned(10, 4) then
              -- no se ha metido ninguna moneda
              null;

            elsif (faltan_reg = to_unsigned(0, 4)) and (pago_excedido = '0') then
              ok_pago <= '1';

            else
              error_pago <= '1';
            end if;

          end if;

        end if;
      end if;
    end if;

    estado_prev <= estado;

    -- AÑADIDO: memorias para detectar flanco
    edge_monedas_prev <= edge_monedas;
    edge_confir_prev  <= edge_confir;

  end if;
end process;

-- Salidas
faltan_monedas_contador <= faltan_reg;  -- 0..10
ok    <= ok_pago;
error <= error_pago;

end Behavioral;
```

3.3.5.2.TESTBENCH



3.3.6. STOCK



La entidad stock se encarga de llevar un control sobre la cantidad de productos de los que se dispone y de llevar un control sobre si la selección que se ha realizado es válida. Se trabaja con nueve productos diferentes, cada uno de ellos con un stock específico, por lo que internamente se utiliza un array stock_mem de nueve posiciones, para almacenar las existencias de cada producto. En el proceso interno se convierte el vector de selección de productos en un valor entero en sel_idx y así se comprueba si la selección es de un único bit a uno, onehot_valid. Una vez confirmado

esto se genera un vector stock_bar_reg donde se encienden tantos bits según unidades queden del producto seleccionado, y estos bits se asignan a los LEDS.

Por otro lado el proceso secuencial funciona de la siguiente manera, cuando se detecta Boton_Confirmar, si la máquina está en el estado de reposo (S0), no hay error de pago y se pulsa el botón de confirmar, se evalúa la selección. Si la selección es inválida (onehot_valid = 0), activa errorProducto. Si hay stock (stock_mem(sel_idx) > 0), decrementa en uno la cantidad de ese producto, activa productoOk y almacena el identificador del producto en producto_id_reg. Si no hay existencias, activa errorProducto. Este registro se limpia al abandonar el estado de reposo o al recibir un reset general. Es decir, una vez pulsado Boton_confirmar tanto si la selección es incorrecta (hayas seleccionado más de un producto a la vez, como si el producto seleccionado no tiene stock) se irá al estado de error. Si tu elección es correcta, se guardará dicha selección para quitar su stock correspondiente.

El diseño permite restaurar las cantidades iniciales a través de la entrada RESET_STOCK.

3.3.6.1.CÓDIGO

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity stock is
    generic (
        NUM_PRODUCTOS : positive := 9;
        PROD_BITS     : positive := 4   -- permite codificar hasta 9 productos
    );
    port(
        clk             : in  std_logic;
        estado          : in  unsigned(2 downto 0);
        productos       : in  std_logic_vector(NUM_PRODUCTOS-1 downto 0);
        error_pago      : in  std_logic;
        RESET_STOCK     : in  std_logic;
        Boton_Confirmar : in  std_logic;

        productoOk     : out std_logic;
        errorProducto  : out std_logic;
        producto_id    : out unsigned(PROD_BITS-1 downto 0);  --prod. determinista
        STOCK_producto_i   : out std_logic_vector(NUM_PRODUCTOS-1 downto 0)
    );
end stock;
```

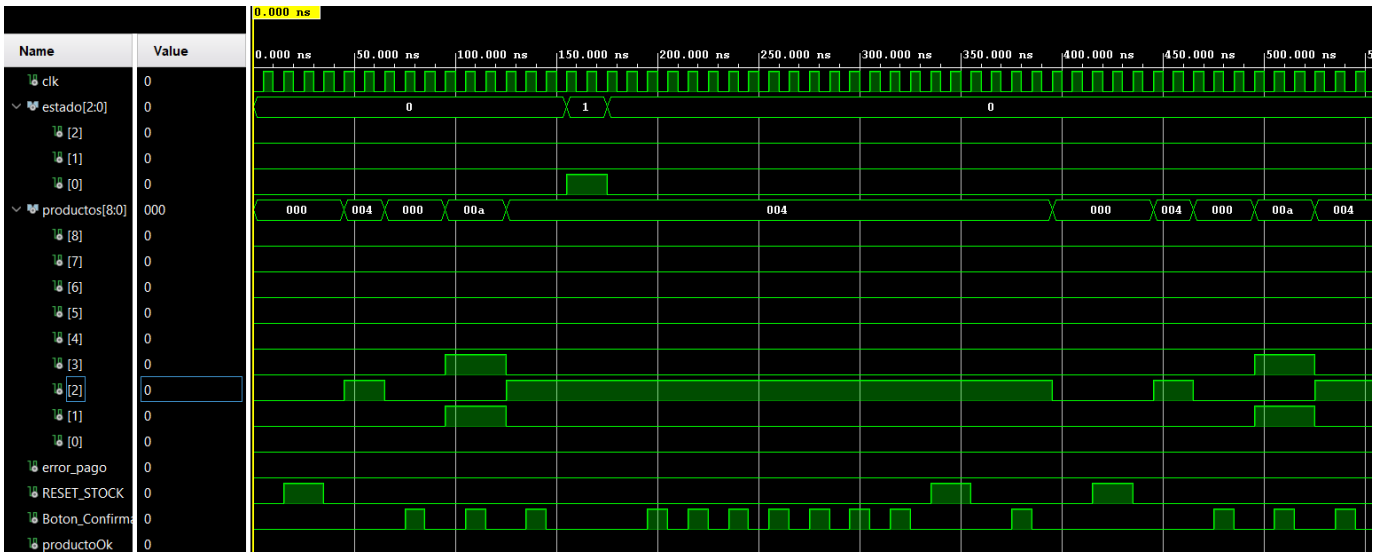
```
architecture Behavioral of stock is

    -- memoria de stock (9 productos)
    type stock_array_t is array (0 to NUM_PRODUCTOS-1) of unsigned(3 downto 0);
```

```
    -- Valores "de fabrica" para poder restaurar con RESET_STOCK
    constant STOCK_INIT : stock_array_t := (
        to_unsigned(5,4),  -- producto 0
        to_unsigned(3,4),  -- producto 1
        to_unsigned(7,4),  -- producto 2
        to_unsigned(3,4),  -- producto 3
        to_unsigned(4,4),  -- producto 4
        to_unsigned(6,4),  -- producto 5
        to_unsigned(3,4),  -- producto 6
        to_unsigned(8,4),  -- producto 7
        to_unsigned(9,4)   -- producto 8
    );

    constant ESTADO_0 : unsigned(2 downto 0) := (others => '0');

    signal stock_mem        : stock_array_t := STOCK_INIT;      -- INICIALIZACION A
VALORES POR DEFECTO stock

    signal productoOk_reg     : std_logic := '0';
    signal errorProducto_reg  : std_logic := '0';

    signal idx               : integer range 0 to NUM_PRODUCTOS-1 := 0;
    signal seleccion_valida  : std_logic := '0';  -- exactamente un bit a '1'

    -- Combinacional: validar one-hot y extraer indice
    signal prod_u        : unsigned(NUM_PRODUCTOS-1 downto 0);
    signal onehot_valid  : std_logic;
    signal sel_idx       : unsigned(PROD_BITS-1 downto 0);

    signal producto_id_reg   : unsigned(PROD_BITS-1 downto 0);  --prod. determinista
    signal estado_prev       : unsigned(2 downto 0) := (others => '0');

    signal stock_bar_reg : std_logic_vector(NUM_PRODUCTOS-1 downto 0) := (others => '0');

begin
    prod_u <= unsigned(productos);

    -- onehot_valid = (productos != 0) AND ((productos AND (productos-1)) = 0)
    onehot_valid <= '1' when (prod_u /= 0) and ((prod_u and (prod_u - 1)) = 0) else '0';

    -- Decodificar indice (combinacional, simple)
    process(prod_u)
        variable tmp : unsigned(PROD_BITS-1 downto 0);
    begin
        tmp := (others => '0');
        for i in 0 to NUM_PRODUCTOS-1 loop
            if prod_u(i) = '1' then
                tmp := to_unsigned(i, PROD_BITS);
            end if;
        end loop;
        sel_idx <= tmp;
    end process;
-------------------------------------------------------------

-- Ejemplos (NUM_PRODUCTOS=9): 0->000000000, 1->100000000, 2->110000000, 3->111000000, 7-
```

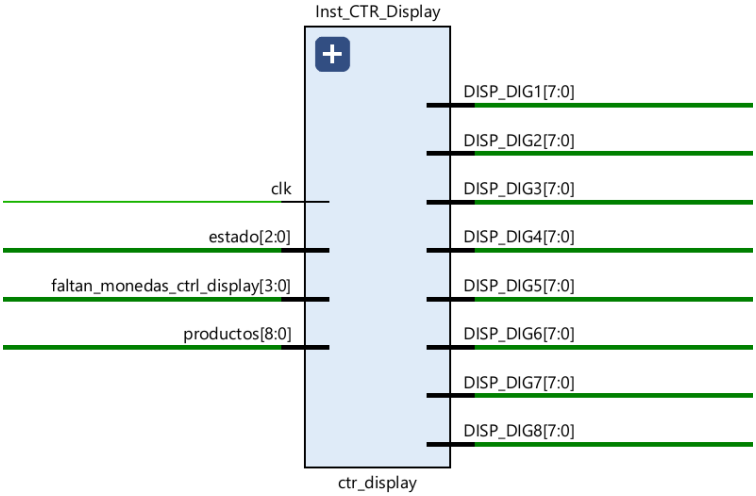
```
>111111100
process(onehot_valid, sel_idx)
    variable nn : integer;
begin
    stock_bar_reg <= (others => '0');
    nn := 0;

    if onehot_valid = '1' then
        nn := to_integer(stock_mem(to_integer(sel_idx)));

        -- Saturar por si el stock fuese mayor que NUM_PRODUCTOS
        if nn > NUM_PRODUCTOS then
            nn := NUM_PRODUCTOS;
        elsif nn < 0 then
            nn := 0;
        end if;

        -- Bucle de rango CONSTANTE (sintetizable)
        for k in 0 to NUM_PRODUCTOS-1 loop
            if k < nn then
                stock_bar_reg(NUM_PRODUCTOS-1-k) <= '1';  -- barra desde MSB
            end if;
        end loop;
    end if;
end process;

-------------------------------------------------------
    -- Secuencial
    process(clk)
    begin

      if rising_edge(clk) then

        -- Pulsos por defecto (1 ciclo)
        productoOk_reg    <= '0';
        errorProducto_reg <= '0';

        -- Detectar re-entrada en estado 0 para limpiar lo mostrado
        if (estado_prev /= ESTADO_0) and (estado = ESTADO_0) then
          producto_id_reg <= to_unsigned(0, PROD_BITS);
        end if;
        estado_prev <= estado;

        -- Reset del stock a valores de fabrica                        -- HE CAMBIADO EL
RESET A 0
        if RESET_STOCK = '0' then
          stock_mem       <= STOCK_INIT;
          producto_id_reg <= to_unsigned(0, PROD_BITS);

        else
          -- Solo trabajamos en estado 0
          if estado = ESTADO_0 then

            -- Si hay error de pago: bloqueo total (no comprar, no tocar stock)
            if error_pago = '1' then
              -- si quieres que no se muestre ningun producto cuando hay error de pago:
```

```
              producto_id_reg <= to_unsigned(0, PROD_BITS);

            else
              -- Solo actuamos si confirman (idealmente esto es un pulso)
              if Boton_Confirmar = '1' then

                -- Seleccion invalida (ninguno o varios) => errorProducto
                if onehot_valid = '0' then
                  errorProducto_reg <= '1';
                else
                  -- Seleccion valida: mirar stock
                  if stock_mem(to_integer(sel_idx)) > 0 then
                    stock_mem(to_integer(sel_idx)) <= stock_mem(to_integer(sel_idx)) - 1;
                    productoOk_reg  <= '1';
                    producto_id_reg <= sel_idx;
                  else
                    errorProducto_reg <= '1';
                  end if;
                end if;
              end if;
            end if;
          end if;
        end if;

  end if;
end process;

    productoOk    <= productoOk_reg;
    errorProducto <= errorProducto_reg;
    producto_id   <= producto_id_reg;


    -- Barra de bits de stock por producto seleccionado
    STOCK_producto_i <= stock_bar_reg;

end Behavioral;
```

3.3.6.2.TESTBENCH



3.3.7. CTR_Display



El módulo ctr_display se encarga de generar los patrones de segmentos necesarios para representar números y texto en un display de siete segmentos. Para cada carácter se define una constante que especifica qué segmentos deben activarse. Por ejemplo, el dígito 1 se codifica con el valor “11111001”, mientras que la letra *r* en minúscula se representa mediante el patrón “10101111”. Además, se incluyen versiones de estas constantes con el punto decimal activado para los casos en los que es necesario mostrarlo.

El funcionamiento es un proceso sincronizado con el reloj, que calcula el contenido de los ocho dígitos del display (d1 a d8) en función del estado actual de la máquina, del vector productos, que me dice que producto se ha seleccionado, y de la cantidad pendiente de pago, representada por la señal faltan_monedas_ctrl_display. Para convertir el código del producto en una combinación de letra y número se emplean comparaciones con patrones de tipo *one-hot*. De este modo, si la señal productos toma el valor “000000001”, se asignan la letra A y el número 0; si vale “000000010”, se muestran la letra A y el número 1; si es “000001000”, se asigna la letra *b* en minúscula junto con el número 0, y así sucesivamente para el resto de combinaciones.

En el estado S0, correspondiente al modo de reposo, el display muestra la palabra “Prod” seguida del identificador del producto seleccionado. Durante el estado S1, asociado al proceso de pago, se presenta el mensaje “P1.0E F”, donde el bloque “P1.0E” indica un precio de un euro y la letra F hace referencia a la cantidad pendiente. Los dos últimos dígitos muestran el importe restante en décimas de euro: el penúltimo dígito representa los euros y el último las décimas. Por ejemplo, si restan 30 céntimos se visualiza “0.3”, y si falta un euro completo se muestra “1.0”. En el estado S2, correspondiente a la entrega del producto, se repite dos veces la palabra “Prod” para enfatizar que la operación ha finalizado correctamente. Por último, en el estado S3 se muestra el mensaje “Erro r _PA”, indicando un error durante la selección o el pago.

Por último, los valores calculados para los dígitos d1 a d8 se asignan a las salidas DISP_DIG1 a DISP_DIG8, que posteriormente son gestionadas por el módulo de multiplexación para activar cada dígito en el momento adecuado.

3.3.7.1.CÓDIGO

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ctr_display is
    generic(
        NUM_SEGS : positive :=8
    );
    port(
        clk : in  std_logic;
        estado          : in  unsigned(2 downto 0);
        productos       : in  std_logic_vector(8 downto 0);
        faltan_monedas_ctrl_display  : in  unsigned(3 downto 0);

        -- Digitos que salen directos al multiplexor
        DISP_DIG1       : out std_logic_vector(NUM_SEGS-1 downto 0);
        DISP_DIG2       : out std_logic_vector(NUM_SEGS-1 downto 0);
        DISP_DIG3       : out std_logic_vector(NUM_SEGS-1 downto 0);
        DISP_DIG4       : out std_logic_vector(NUM_SEGS-1 downto 0);
        DISP_DIG5       : out std_logic_vector(NUM_SEGS-1 downto 0);
        DISP_DIG6       : out std_logic_vector(NUM_SEGS-1 downto 0);
        DISP_DIG7       : out std_logic_vector(NUM_SEGS-1 downto 0);
```

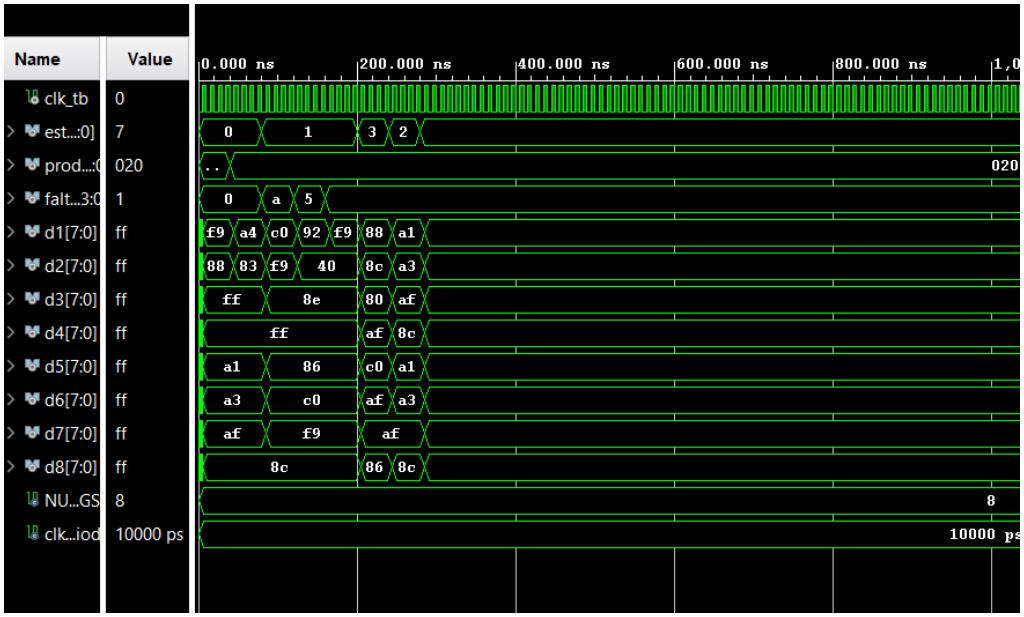
```
        DISP_DIG8       : out std_logic_vector(NUM_SEGS-1 downto 0)
    );

--   DISPLAY
--  [Punto,g,f,e,d,c,b,a]   <--Lectura del vector
--  [8,7,6,5,4,3,2,1]       <-- Lectura de dígitos

--          CONSTANTES LETRAS
constant Const_Letra_OFF : std_logic_vector(7 downto 0) := "11111111";  -- Apagado
constant Const_Letra_A   : std_logic_vector(7 downto 0) := "10001000";        -- A
constant Const_Letra_E   : std_logic_vector(7 downto 0) := "10000110";        -- E
constant Const_Letra_F   : std_logic_vector(7 downto 0) := "10001110";        -- F
constant Const_Letra_r   : std_logic_vector(7 downto 0) := "10101111";        -- r
(minúscula)
constant Const_Letra_d   : std_logic_vector(7 downto 0) := "10100001";        -- d
(minúscula)
constant Const_Letra_b   : std_logic_vector(7 downto 0) := "10000011";        -- b
(minúscula)
constant Const_Letra_c   : std_logic_vector(7 downto 0) := "10100111";        -- c
(minúscula)
constant Const_Letra_C_ma: std_logic_vector(7 downto 0) := "11000110";        -- C
(mayúscula)
constant Const_Letra_O_ma: std_logic_vector(7 downto 0) := "11000000";        -- O
(mayúscula)
constant Const_Letra_o   : std_logic_vector(7 downto 0) := "10100011";        -- o
(minúscula)
constant Const_Letra_P   : std_logic_vector(7 downto 0) := "10001100";        -- P
constant Const_Letra_X   : std_logic_vector(7 downto 0) := "10110110";        -- X
(aproximación)
constant Const_Letra_Euro: std_logic_vector(7 downto 0) := "10000111";        -- € (euro,
aproximación estándar)
constant Const_BarraBaja : std_logic_vector(7 downto 0) := "10000000";        -- 0
-- NÚMEROS
constant Const_Num_0     : std_logic_vector(7 downto 0) := "11000000";        -- 0
constant Const_Num_0_punto:std_logic_vector(7 downto 0) := "01000000";        -- 0.
constant Const_Num_1     : std_logic_vector(7 downto 0) := "11111001";        -- 1
constant Const_Num_1_punto:std_logic_vector(7 downto 0) := "11111001";        -- 1.
constant Const_Num_2     : std_logic_vector(7 downto 0) := "10100100";        -- 2
constant Const_Num_3     : std_logic_vector(7 downto 0) := "10110000";        -- 3
constant Const_Num_4     : std_logic_vector(7 downto 0) := "10011001";        -- 4
constant Const_Num_5     : std_logic_vector(7 downto 0) := "10010010";        -- 5
constant Const_Num_6     : std_logic_vector(7 downto 0) := "10000010";        -- 6
constant Const_Num_7     : std_logic_vector(7 downto 0) := "11111000";        -- 7
constant Const_Num_8     : std_logic_vector(7 downto 0) := "10000000";        -- 8
constant Const_Num_9     : std_logic_vector(7 downto 0) := "10010000";        -- 9




end ctr_display;

architecture Behavioral of ctr_display is
begin
-------------------------------------------------------------------------------------------
-------- QUE SE INICIALICEN A 0 SOLO AL PRINCIPIO
    process(clk)
```

```
        variable d1, d2, d3, d4, d5, d6, d7, d8 : std_logic_vector(NUM_SEGS-1 downto 0);
        variable prod_letra : std_logic_vector(NUM_SEGS-1 downto 0);
        variable prod_num   : std_logic_vector(NUM_SEGS-1 downto 0);
        variable monedas_centenas : std_logic_vector(NUM_SEGS-1 downto 0);
        variable monedas_decenas  : std_logic_vector(NUM_SEGS-1 downto 0);
-----------------------------------------------------------------------------------------
---------------------

        -- Según producto descodificamos:
        --     A   B   C
        --    000 000 000
        --    012 012 012
    begin
    if rising_edge(clk) then
         if productos = "000000001" then
            prod_letra := Const_Letra_A;   -- A
            prod_num   := Const_Num_0;     -- 0
        elsif productos = "000000010" then
            prod_letra := Const_Letra_A;   -- A
            prod_num   := Const_Num_1;     -- 1
        elsif productos = "000000100" then
            prod_letra := Const_Letra_A;   -- A
            prod_num   := Const_Num_2;     -- 2
        elsif productos = "000001000" then
            prod_letra := Const_Letra_b;   -- b
            prod_num   := Const_Num_0;     -- 0
        elsif productos = "000010000" then
            prod_letra := Const_Letra_b;   -- b
            prod_num   := Const_Num_1;     -- 1
        elsif productos = "000100000" then
            prod_letra := Const_Letra_b;   -- b
            prod_num   := Const_Num_2;     -- 2
        elsif productos = "001000000" then
            prod_letra := Const_Letra_C_ma; -- C
            prod_num   := Const_Num_0;      -- 0
        elsif productos = "010000000" then
            prod_letra := Const_Letra_C_ma; -- C
            prod_num   := Const_Num_1;      -- 1
        elsif productos = "100000000" then
            prod_letra := Const_Letra_C_ma; -- C
            prod_num   := Const_Num_2;      -- 2
        else
            prod_letra := Const_Letra_OFF;
            prod_num   := Const_Letra_OFF;
        end if;

-- MONEDAS, en un vector de 2 bits, para poner euros y se cuenta en número de monedas de
10 cents:
        case faltan_monedas_ctrl_display is

    when to_unsigned(0,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_0;        -- 0   -> 0.0
    when to_unsigned(1,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_1;        -- 1   -> 0.1
```

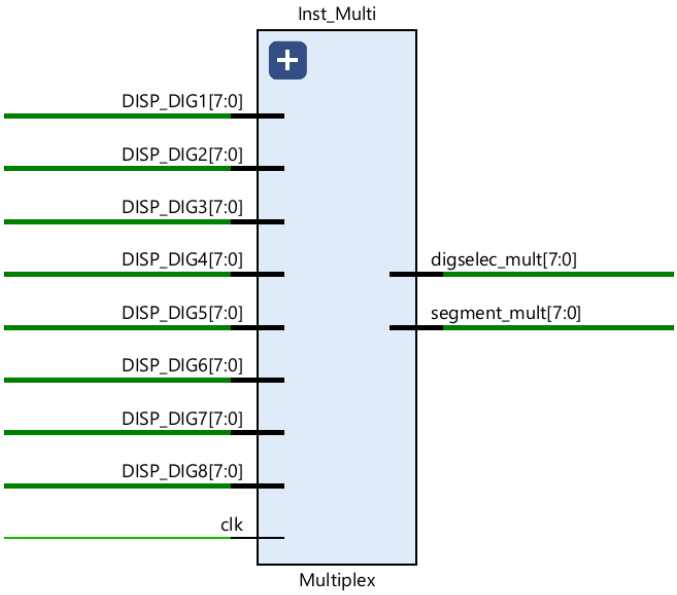
```
    when to_unsigned(2,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_2;        -- 2   -> 0.2
    when to_unsigned(3,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_3;        -- 3   -> 0.3
    when to_unsigned(4,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_4;        -- 4   -> 0.4
    when to_unsigned(5,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_5;        -- 5   -> 0.5
    when to_unsigned(6,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_6;        -- 6   -> 0.6
    when to_unsigned(7,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_7;        -- 7   -> 0.7
    when to_unsigned(8,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_8;        -- 8   -> 0.8
    when to_unsigned(9,4) =>
        monedas_centenas := Const_Num_0_punto;  -- 0.
        monedas_decenas  := Const_Num_9;        -- 9   -> 0.9
    when to_unsigned(10,4) =>
        monedas_centenas := Const_Num_1;        -- 1
        monedas_decenas  := Const_Num_0;        -- 0   -> 1.0
    when others =>
        monedas_centenas := (others => '1');   -- apagado / error
        monedas_decenas  := (others => '1');
end case;

        -- ESTADOS, segun ello sacamos una cosa u otra por el multiplexor
        if estado = "000" then      -- DISPLAY:  PrOd  <producto>

            --[Punto,g,f,e,d,c,b,a]<--Lectura del vector
            --[8,7,6,5,4,3,2,1] <--- Lectura de dígitos
            d8 := Const_Letra_P; --P
            d7 := Const_Letra_r; --r
            d6 := Const_Letra_o; --O
            d5 := Const_Letra_D; --d
            d4 := Const_Letra_OFF; --apagado
            d3 := Const_Letra_OFF; --apagado
            d2 := prod_letra;
            d1 := prod_num;

        elsif estado = "001" then   -- DISPLAY:  P1.0E  F <faltan>

            d8 := Const_Letra_P;
            d7 := Const_Num_1;
            d6 := Const_Num_0;        -- el punto lo gestiona el multiplexor si procede
            d5 := Const_Letra_E;
            d4 := Const_Letra_OFF;
            d3 := Const_Letra_F;
            d2 := monedas_centenas;  -- o decenas/unidades según tu diseño
            d1 := monedas_decenas;
```

```
        elsif estado = "010" then   -- DISPLAY:  EXP _  P r O d

            d8 := Const_Letra_P;        -- P
            d7 := Const_Letra_r;        -- R
            d6 := Const_Letra_O;        -- O
            d5 := Const_Letra_d;        -- d
            d4 := Const_Letra_P;        -- P
            d3 := Const_Letra_r;        -- r
            d2 := Const_Letra_O;        -- O
            d1 := Const_Letra_d;        -- d

        elsif estado = "011" then   -- DISPLAY:  E r r O r _ P A

            d8 := Const_Letra_E;        -- E
            d7 := Const_Letra_r;        -- r
            d6 := Const_Letra_r;        -- r
            d5 := Const_Letra_O_ma;     -- O
            d4 := Const_Letra_r;        -- r
            d3 := Const_BarraBaja;      -- _
            d2 := Const_Letra_P;        -- P
            d1 := Const_Letra_A;        -- A


        else                        -- DISPLAY:  N O _ H  P r O d           <<<<<<<<NO SE
CUMPLE ESTO
            d1 := Const_Letra_OFF;
            d2 := Const_Letra_OFF;
            d3 := Const_Letra_OFF;
            d4 := Const_Letra_OFF;
            d5 := Const_Letra_OFF;
            d6 := Const_Letra_OFF;
            d7 := Const_Letra_OFF;
            d8 := Const_Letra_OFF;
        end if;

        DISP_DIG1 <= d1;
        DISP_DIG2 <= d2;
        DISP_DIG3 <= d3;
        DISP_DIG4 <= d4;
        DISP_DIG5 <= d5;
        DISP_DIG6 <= d6;
        DISP_DIG7 <= d7;
        DISP_DIG8 <= d8;

        end if;
    end process;

end Behavioral;
```

3.3.7.2.TESTBENCH



3.3.8. Multiplex



La entidad Multiplex se encarga de controlar la visualización en un display de siete segmentos formado por ocho dígitos. El funcionamiento se basa en encender únicamente un dígito en cada instante, mientras se envía al bus común el patrón de segmentos que le corresponde. Tras un breve intervalo, ese dígito se apaga y se activa el siguiente, repitiéndose este proceso de forma cíclica.

El control de la multiplexación se realiza mediante un contador de tres bits, denominado sel, que se incrementa en cada flanco de subida del reloj. En función de

su valor, una estructura *case* selecciona uno de los ocho patrones de entrada (desde DISP_DIG8 hasta DISP_DIG1) y lo envía a la salida segment_mult. De forma simultánea, se genera la señal digselec_mult, que habilita únicamente el dígito activo, utilizando una lógica activa a nivel bajo. Así, cuando sel toma el valor “000”, se activa el dígito correspondiente al canal 7 y se muestra el contenido de DISP_DIG8; cuando vale “001”, se activa el siguiente dígito y se carga DISP_DIG7, y así sucesivamente.

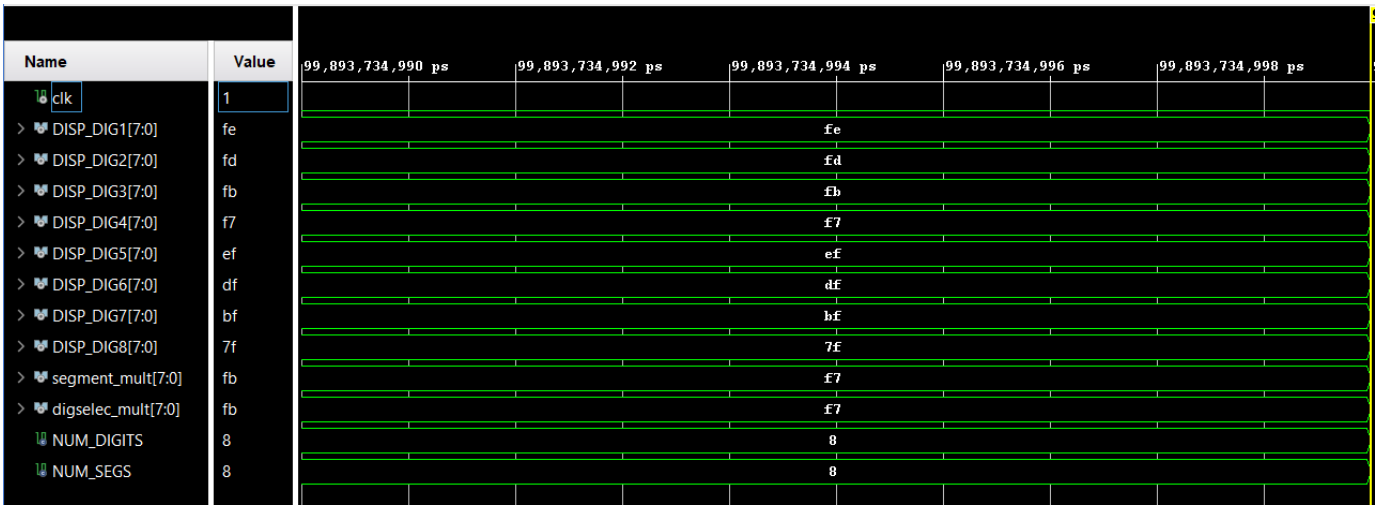
Con esta técnica lo que se consigue es mostrar números y mensajes completos en los ocho dígitos del display utilizando un único conjunto de líneas para los segmentos, simplificando así el cableado y los recursos del sistema.

3.3.8.1.CÓDIGO

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Multiplex is
    generic (
        NUM_DIGITS : positive := 8;
        NUM_SEGS   : positive := 8  -- 7 segmentos + punto (bit 0)
    );
    port (
        clk           : in  STD_LOGIC;

        DISP_DIG1     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG2     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG3     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG4     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG5     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG6     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG7     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
        DISP_DIG8     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);

        segment_mult  : out STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0); -- el 7 es el punto
del digito
        digselec_mult : out STD_LOGIC_VECTOR(NUM_DIGITS-1 downto 0) -- an[0..7] (active-
low)
    );
end Multiplex;

architecture Behavioral of Multiplex is
    signal sel : unsigned(2 downto 0) := (others => '0');
begin

    process(clk)
    begin
        if rising_edge(clk) then

            sel <= sel + 1;

            case sel is
                when "000" =>
                    segment_mult  <= DISP_DIG8;
```

```
                    digselec_mult <= "01111111";  -- AN7 activo (0)

                when "001" =>
                    segment_mult  <= DISP_DIG7;
                    digselec_mult <= "10111111";  -- AN6 activo (0)

                when "010" =>
                    segment_mult  <= DISP_DIG6;
                    digselec_mult <= "11011111";  -- AN5 activo (0)

                when "011" =>
                    segment_mult  <= DISP_DIG5;
                    digselec_mult <= "11101111";  -- AN4 activo (0)

                when "100" =>
                    segment_mult  <= DISP_DIG4;
                    digselec_mult <= "11110111";  -- AN3 activo (0)

                when "101" =>
                    segment_mult  <= DISP_DIG3;
                    digselec_mult <= "11111011";  -- AN2 activo (0)

                when "110" =>
                    segment_mult  <= DISP_DIG2;
                    digselec_mult <= "11111101";  -- AN1 activo (0)

                when others =>
                    segment_mult  <= DISP_DIG1;
                    digselec_mult <= "11111110";  -- AN0 activo (0)
            end case;

        end if;
    end process;

end Behavioral;
```

3.3.8.2.TESTBENCH



3.3.9. TOP

La entidad **TOP** actúa como el bloque principal del sistema, se encarga de integrar y coordinar el funcionamiento de todos los módulos. En ella se definen los puertos de entrada y salida que se conectan directamente a la placa, así como las señales internas necesarias para la comunicación entre las distintas entidades. Por tanto, en el top se instancian los módulos del prescaler, sincronizador, máquina de estados, edgedetector, el contador de monedas, el stock, el controlador del display y el multiplexor.

Además, en esta entidad se genera la señal en_leds, que determina cuándo debe mostrarse la información de stock en el vector Err. Cuando esta señal está activa, los bits del vector representan el stock disponible de cada producto; en caso contrario, dichos bits permanecen a cero. Junto a esta información, se asignan otros bits para indicar el estado actual de la máquina, de modo que el conjunto de LEDs proporcionan una visión rápida del stock, los estados de funcionamiento y posibles situaciones de error.

3.3.9.1.CÓDIGO

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;


-- Uncomment the following library declaration if using
```

```
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity TOP is
generic (
        Monedas       : POSITIVE := 4;   -- genérico con valor por defecto 4
        NUM_PRODUCTOS : POSITIVE := 9;   -- genérico con valor por defecto 9
        NUM_DIGITS    : positive := 8;
        NUM_SEGS      : positive := 8;   -- antes 7, ahora 8 (anodos + punto)
        NUM_PULSOS_TOP: positive := 5
    );

Port (
           button_confir: in STD_LOGIC; --Confirmación de pedido
           vector_botones_monedas : in std_logic_vector (MONEDAS-1 downto 0);


           digsel : in STD_LOGIC_VECTOR (8 downto 0); --Los 9 productos que tenemos
(switches)

           CLK    : in std_logic;
           RST    : in STD_LOGIC;
           digselec      : out std_logic_vector(NUM_DIGITS-1 downto 0);
           segment       : out STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);

           ESTADO_OFF_button    : in STD_LOGIC;
           control_LEDS_1       : in STD_LOGIC;
           control_LEDS_2       : in STD_LOGIC;


           Err : out STD_LOGIC_VECTOR (15 downto 0)
           );
end TOP;

architecture Behavioral of TOP is

    signal clk_out : std_logic;

    -- SEÑALES DEL FSM
    signal estado_s       : unsigned(2 downto 0);


    -- SEÑALES MULTIPLEXOR     -------------------------------------------
    -- SEÑALES PARA LOS DIGITOS DEL DISPLAY, de tamaño 8 bits porque el último es el
punto del dígito.
    -- Sirven como nexo entre el CTRL_DISPLAY y el Multiplexor que es quien se comunica
con el HW
        signal DISP_DIG1_s    : std_logic_vector(NUM_SEGS-1 downto 0);
        signal DISP_DIG2_s    : std_logic_vector(NUM_SEGS-1 downto 0);
        signal DISP_DIG3_s    : std_logic_vector(NUM_SEGS-1 downto 0);
        signal DISP_DIG4_s    : std_logic_vector(NUM_SEGS-1 downto 0);
```

```
        signal DISP_DIG5_s    : std_logic_vector(NUM_SEGS-1 downto 0);
        signal DISP_DIG6_s    : std_logic_vector(NUM_SEGS-1 downto 0);
        signal DISP_DIG7_s    : std_logic_vector(NUM_SEGS-1 downto 0);
        signal DISP_DIG8_s    : std_logic_vector(NUM_SEGS-1 downto 0);
    --Vector de los digitos síncrono, que permite al multiplexor decirle al HW qué digito
activa por señal temporal
        signal dig_sel_sinc: STD_LOGIC_VECTOR (8 downto 0);


    -- Señales SINCRONIZADOR salida
    signal vector_productos_sinc       : std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
    signal vector_monedas_sinc         : std_logic_vector(Monedas-1 downto 0);
    signal boton_PAGO_sinc             : std_logic;

    -- Señales EDGEDTCTR salida
    signal edge_monedas_s              : std_logic;
    signal vector_monedas_deter_s      : std_logic_vector(Monedas-1 downto 0);
    signal edge_boton_pago_s           : std_logic;

    -- Señales Contador salida
    signal faltan_monedas_s : unsigned(3 downto 0);
    signal ok_s, error_s       : std_logic;

    -- Señales STOCK salida
    signal productoOk_s     : std_logic;
    signal errorProducto_s  : std_logic;
    signal producto_id_s    : unsigned(3 downto 0);

    signal STOCK_producto_i_s :  std_logic_vector(NUM_PRODUCTOS-1 downto 0);

    signal en_leds : std_logic; ------------    ----------------------------------------
---------------------------------------- OJO
    -- Señales Máquina de estados salida
    signal reset_general  : std_logic;


COMPONENT Prescaler
port(
        clk_in  : in  std_logic;   -- 100 MHz
        rst     : in  std_logic;   -- Reset síncrono
        clk_out : out std_logic    -- 16 kHz
    );
END COMPONENT;

COMPONENT Sincronizador
 Generic(
        NUM_PRODUCTOS: POSITIVE;
        NUM_MONEDAS: POSITIVE
    );
 port(
        CLK: in std_logic;

        AS_PRODUCTOS: in std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
        AS_MONEDAS: in std_logic_vector (NUM_MONEDAS - 1 downto 0);
        AS_CONFI : in std_logic;
```

```
        S_PRODUCTOS: out std_logic_vector (NUM_PRODUCTOS - 1 downto 0);
        S_MONEDAS: out std_logic_vector (NUM_MONEDAS - 1 downto 0);
        S_CONFI : out std_logic
    );
END COMPONENT;

component EDGEDTCTR is
        Generic(
            NUM_MONEDAS  : positive;
            NUM_PULSOS   : positive
        );
        Port(
            CLK : in STD_LOGIC;

            SYNC_IN_MONEDAS        : in STD_LOGIC_VECTOR (NUM_MONEDAS -1 downto 0);
            SYNC_IN_CONFIRMACION   : in STD_LOGIC;
            RESET_EDGE_DTC         : in STD_LOGIC;

            EDGE_MONEDAS_DETECTADO : out STD_LOGIC;
            EDGE_BOTON_PAGO        : out STD_LOGIC;

            Vector_monedas_deter   : out STD_LOGIC_VECTOR (NUM_MONEDAS -1 downto 0)
        );
    end component;

    component contador_monedas is
        port(
            clk            : in  std_logic;
            edge_monedas   : in  std_logic;
            edge_confir    : in  std_logic;
            monedas        : in  std_logic_vector(3 downto 0);
            estado         : in  unsigned(2 downto 0);
            Reset_Contador : in std_logic ;

            faltan_monedas_contador : out unsigned(3 downto 0);
            ok             : out std_logic;
            error          : out std_logic
        );
    end component;

component stock is
        generic (
            NUM_PRODUCTOS : positive := 9;
            PROD_BITS     : positive := 4
        );
        port(
            clk            : in  std_logic;
            estado         : in  unsigned(2 downto 0);
            productos      : in  std_logic_vector(NUM_PRODUCTOS-1 downto 0);
            error_pago     : in  std_logic;
            RESET_STOCK    : in  std_logic;
            Boton_Confirmar : in  std_logic;


            productoOk     : out std_logic;
            errorProducto  : out std_logic;
```

```
            producto_id     : out unsigned(PROD_BITS-1 downto 0);
            STOCK_producto_i    : out std_logic_vector(NUM_PRODUCTOS-1 downto 0)

        );
    end component;

COMPONENT Maq_Estados is
    Port(
            Error_producto      : in std_logic;
            Error_Pago          : in std_logic;
            OK_Pago             : in std_logic;
            OK_Producto         : in std_logic;
            RESET               : in std_logic;
            CLK                 : in std_logic;
            ESTADO_OFF          : in std_logic;

            RESET_SALIDA :  out std_logic;
            ESTADO_SALIDA : out unsigned(2 downto 0);
            LEDS : out STD_LOGIC_VECTOR (6 downto 0)

            );
END COMPONENT;

 component Multiplex is
        generic (
            NUM_DIGITS :positive ;
            NUM_SEGS   :positive
        );
        port (
            clk           : in  STD_LOGIC;

            DISP_DIG1     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG2     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG3     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG4     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG5     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG6     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG7     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            DISP_DIG8     : in  STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);

            segment_mult  : out STD_LOGIC_VECTOR(NUM_SEGS-1 downto 0);
            digselec_mult : out STD_LOGIC_VECTOR(NUM_DIGITS-1 downto 0)
        );
end component;

 component ctr_display is
        generic(
            NUM_SEGS : positive
        );
        port(
            clk : in  std_logic;
            estado         : in  unsigned(2 downto 0);
            productos      : in  std_logic_vector(8 downto 0);
            faltan_monedas_ctrl_display : in  unsigned(3 downto 0);

            DISP_DIG1      : out std_logic_vector(NUM_SEGS-1 downto 0);
```

```
            DISP_DIG2      : out std_logic_vector(NUM_SEGS-1 downto 0);
            DISP_DIG3      : out std_logic_vector(NUM_SEGS-1 downto 0);
            DISP_DIG4      : out std_logic_vector(NUM_SEGS-1 downto 0);
            DISP_DIG5      : out std_logic_vector(NUM_SEGS-1 downto 0);
            DISP_DIG6      : out std_logic_vector(NUM_SEGS-1 downto 0);
            DISP_DIG7      : out std_logic_vector(NUM_SEGS-1 downto 0);
            DISP_DIG8      : out std_logic_vector(NUM_SEGS-1 downto 0)
        );
    end component;




begin

Inst_Prescaler : Prescaler
    port map(
        clk_in  => clk,
        rst    => rst,
        clk_out => clk_out
    );

   -- SINCRONIZADOR

Inst_Sincronizador : Sincronizador
        generic map(
            NUM_PRODUCTOS => NUM_PRODUCTOS,
            NUM_MONEDAS   => Monedas
        )
        port map(
            CLK           => clk_out,
            AS_PRODUCTOS => digsel,
            AS_MONEDAS    => vector_botones_monedas,
            AS_CONFI      => button_confir,
            S_PRODUCTOS   => dig_sel_sinc,
            S_MONEDAS     => vector_monedas_sinc,
            S_CONFI       => boton_PAGO_sinc
        );

    Inst_Maq_Estados : Maq_Estados
    Port Map(
            Error_producto  => errorProducto_s,
            Error_Pago      => error_s,
            OK_Pago         => ok_s,
            OK_Producto     => productoOk_s,
            RESET           => rst,
            CLK             => clk_out,
            ESTADO_OFF      => ESTADO_OFF_button,

            RESET_SALIDA    => reset_general,
            ESTADO_SALIDA   => estado_s,
            LEDS(0)         => Err(9),
            LEDS(1)         => Err(10),
            LEDS(2)         => Err(11),
            LEDS(3)         => Err(12),
            LEDS(4)         => Err(13),
```

```
        LEDS(5)         => Err(14),
        LEDS(6)         => Err(15)
        );

-- EDGE DETECTOR
inst_EDGE_DETECT : EDGEDTCTR
    generic map(
        NUM_MONEDAS => 4,
        NUM_PULSOS => NUM_PULSOS_TOP
    )
    port map(
        CLK                     => clk_out,
        SYNC_IN_MONEDAS         => vector_monedas_sinc,
        SYNC_IN_CONFIRMACION    => boton_PAGO_sinc,
        RESET_EDGE_DTC          => reset_general,

        EDGE_MONEDAS_DETECTADO => edge_monedas_s,
        EDGE_BOTON_PAGO         => edge_boton_pago_s,
        Vector_monedas_deter    => vector_monedas_deter_s
    );

-- CONTADOR MONEDAS
Inst_Contador : contador_monedas
    port map(
        clk             => clk_out,
        edge_monedas    => edge_monedas_s,
        edge_confir     => edge_boton_pago_s,
        monedas         => vector_monedas_deter_s,
        estado          => estado_s,
        Reset_Contador => reset_general,

        faltan_monedas_contador => faltan_monedas_s,
        ok              => ok_s,
        error           => error_s
    );

    Inst_Stock : stock
    generic map(
        NUM_PRODUCTOS => 9,
        PROD_BITS     => 4
    )
    port map(
        clk             => clk_out,
        estado          => estado_s,
        productos       => dig_sel_sinc,
        error_pago      => error_s,
        RESET_STOCK     => reset_general,
        Boton_Confirmar => edge_boton_pago_s,


        productoOk      => productoOk_s,
        errorProducto   => errorProducto_s,
        producto_id     => producto_id_s,
        STOCK_producto_i => STOCK_producto_i_s
    );
```

```
    Inst_CTR_Display : ctr_display
        generic map(
            NUM_SEGS => NUM_SEGS
        )
        port map(
            clk             => clk_out,
            estado           => estado_s,
            productos        => dig_sel_sinc,
            faltan_monedas_ctrl_display  => faltan_monedas_s,

            DISP_DIG1        => DISP_DIG1_s,
            DISP_DIG2        => DISP_DIG2_s,
            DISP_DIG3        => DISP_DIG3_s,
            DISP_DIG4        => DISP_DIG4_s,
            DISP_DIG5        => DISP_DIG5_s,
            DISP_DIG6        => DISP_DIG6_s,
            DISP_DIG7        => DISP_DIG7_s,
            DISP_DIG8        => DISP_DIG8_s
        );


Inst_Multi : Multiplex
        generic map(
            NUM_DIGITS => NUM_DIGITS,
            NUM_SEGS   => NUM_SEGS
        )
        port map(
            clk            => clk_out,

            DISP_DIG1      => DISP_DIG1_s,
            DISP_DIG2      => DISP_DIG2_s,
            DISP_DIG3      => DISP_DIG3_s,
            DISP_DIG4      => DISP_DIG4_s,
            DISP_DIG5      => DISP_DIG5_s,
            DISP_DIG6      => DISP_DIG6_s,
            DISP_DIG7      => DISP_DIG7_s,
            DISP_DIG8      => DISP_DIG8_s,

            segment_mult   => segment,
            digselec_mult => digselec
        );

en_leds <= control_LEDS_1 xor control_LEDS_2;

Err(0) <= STOCK_producto_i_s(0) when en_leds = '1' else '0';
Err(1) <= STOCK_producto_i_s(1) when en_leds = '1' else '0';
Err(2) <= STOCK_producto_i_s(2) when en_leds = '1' else '0';
Err(3) <= STOCK_producto_i_s(3) when en_leds = '1' else '0';
Err(4) <= STOCK_producto_i_s(4) when en_leds = '1' else '0';
Err(5) <= STOCK_producto_i_s(5) when en_leds = '1' else '0';
Err(6) <= STOCK_producto_i_s(6) when en_leds = '1' else '0';
Err(7) <= STOCK_producto_i_s(7) when en_leds = '1' else '0';
Err(8) <= STOCK_producto_i_s(8) when en_leds = '1' else '0';
```

```
end Behavioral;
```

4. Github

https://github.com/GuillermoRosale5/Trabajo_FPGA

5. Conclusión

Durante la realización de este trabajo hemos logrado aprender en profundidad el funcionamiento de una FPGA. No solo eso, si no que hemos mejorado nuestras capacidades de diseño y estructuración debido a que nos hemos visto forzado a ello para poder llevar a cabo el proyecto que teníamos en mente.

El proyecto ha logrado cumplir con cada una de los objetivos propuestos. Una de las cualidades de este proyecto que más nos ha demandado ha sido la modularidad de los bloques. El ser capaces de simplificar las ideas funcionales y plasmarlas en bloques individuales con la menor cantidad de código posible, más luego la conexión entre dichos bloques. Esta cualidad del proyecto nos ha llevado a reformar entidades enteras y dividir entidades en dos entidades más pequeñas, lo cual ha sido muy demandante debido a que era un trabajo considerable y no se sentía que avanzara el proyecto con estos cambios.

En cuanto a mejoras a añadir, nos hubiera gustado haber añadido un precio diferente para cada uno de los 9 productos y haber modificado como pasa el módulo del contador la información al CTR_Display, debido a que se carga la operación de las monedas pero no se muestra dicha operación en el display hasta que no se introduce otra moneda nueva.

Ambos son cambios que no afectan al funcionamiento de la máquina expendedora pero que nos hubiera gustado introducir en caso de poder dedicarle más tiempo al trabajo.

Por último, viendo que se han cumplido todos los objetivos y que hemos logrado obtener una máquina expendedora funcional, podemos dar como concluido el proyecto de manera satisfactoria.